deeplearning.ai

Case Studies

Why look at case studies?

Outline

Classic networks:

- LeNet-5 ←
- AlexNet ←
- VGG ←

ResNet (152) # layers

Inception

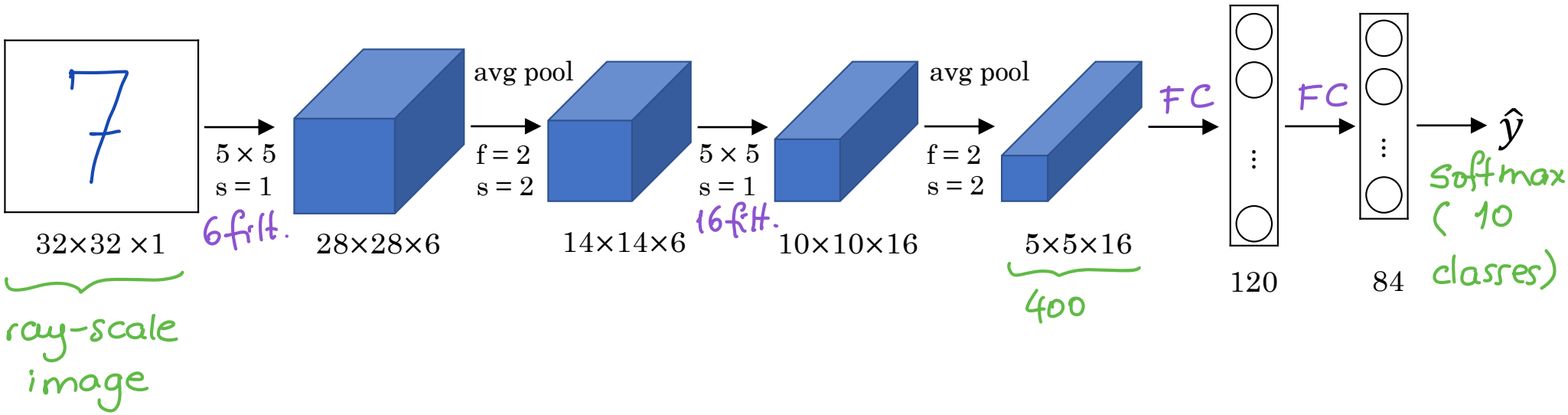
- A good way to gain intuition about CNNs is to read or see other examples of CNNs.
- A CNN architecture that works well on one computer vision task often works well on other tasks as well.
- The ideas in these networks may also be useful in other applications other than computer vision.

deeplearning.ai

Case Studies

Classic networks

LeNet - 5 (Recognize hand-written digits)



Small Network : ~60K parameters

As we go from left to right: $n_H, n_W \downarrow$, $n_c \uparrow$ (increases)

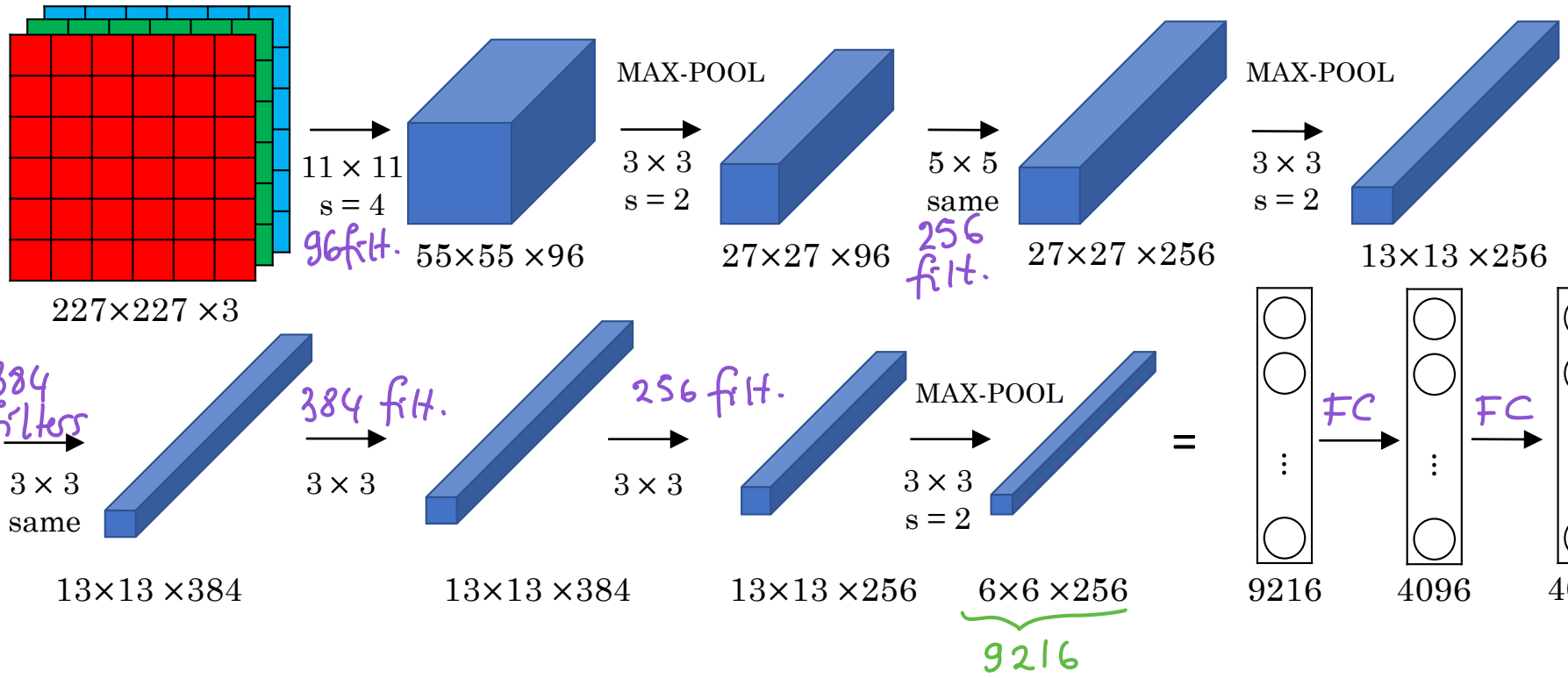
Architecture : conv – pool – conv – pool – fc – fc – output (common)

Advanced comments: Activation functions: sigmoid / tanh (not ReLU)

[LeCun et al., 1998. Gradient-based learning applied to document recognition]

Andrew Ng

AlexNet



[Krizhevsky et al., 2012. ImageNet classification with deep convolutional neural networks]

Hinton

Andrew Ng

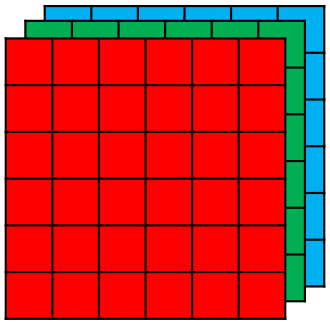
- AlexNet is similar to LeNet but it is much bigger.
- AlexNet has ~ 60M parameters.
- Activation function is ReLU.
- It had a complicated way of using multiple GPUs.
- This was the first paper that convinced the computer vision community that deep learning really works well in computer vision.
- AlexNet has a lot of hyperparameters.

VGG - 16

CONV = 3×3 filter, s = 1, same

MAX-POOL = 2×2 , s = 2

} same in the architecture below



224×224 ×3

→ 224×224×64 [CONV 64] ×2 → POOL → 112×112 ×64 → [CONV 128] ×2 → 112×112 ×128 → POOL → 56×56 ×128

filters

layers

→ 56×56 ×256 [CONV 256] ×3 → POOL → 28×28 ×256 → [CONV 512] ×3 → 28×28 ×512 → POOL → 14×14×512

→ 14×14 ×512 [CONV 512] ×3 → POOL → 7×7×512 → FC 4096 → FC 4096 → Softmax 1000

[Simonyan & Zisserman 2015. Very deep convolutional networks for large-scale image recognition]

Andrew Ng

Properties of VGG-16:

- Instead of using many hyperparameters, focuser on conv layers with 3x3 filters, stride 1, and same padding.
- Pooling layers have filters which are 2x2 with a stride of 2.
- This simplified the CNN architecture.
- VGG-16 has 16 layers that have weights.
- # parameters: ~138 M
- Architecture is quite uniform.
- The # of filters were doubled as you go deeper in the network (64 → 128 → 256 → 512) whereas height and width is halved (224 → 112 → 56 → 28 → 14 → 7) systematically.
- Down-side: pretty large to train.
- VGG-19: a larger version.

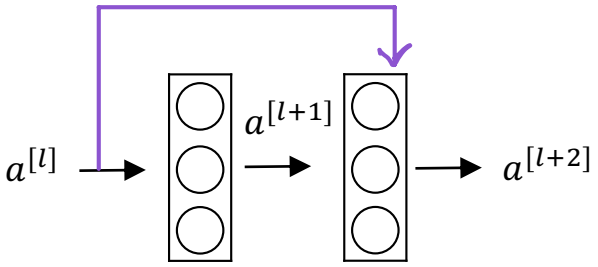
deeplearning.ai

Case Studies

Residual Networks (ResNets)

- Very deep networks are difficult to train because of the vanishing and exploding gradient problems.
- Skip connections taker the activations in one layer and feeds it to another layer much deeper in the neural network.
- This enables ResNet to train very, very deep networks (over 100 layers)

Residual block



Two layers of a network

"short cut" / skip connection

$a^{[l]}$ → Linear → ReLU → $a^{[l+1]}$ → Linear → ReLU → $a^{[l+2]}$

main path

$$z^{[l+1]} = W^{[l+1]}\, a^{[l]} + b^{[l+1]} \qquad a^{[l+1]} = g(z^{[l+1]})$$

$$z^{[l+2]} = W^{[l+2]} a^{[l+1]} + b^{[l+2]} \qquad a^{[l+2]} = g(z^{[l+2]})$$

$$a^{[l+2]} = g\left(z^{[l+2]} + a^{[l]}\right)$$

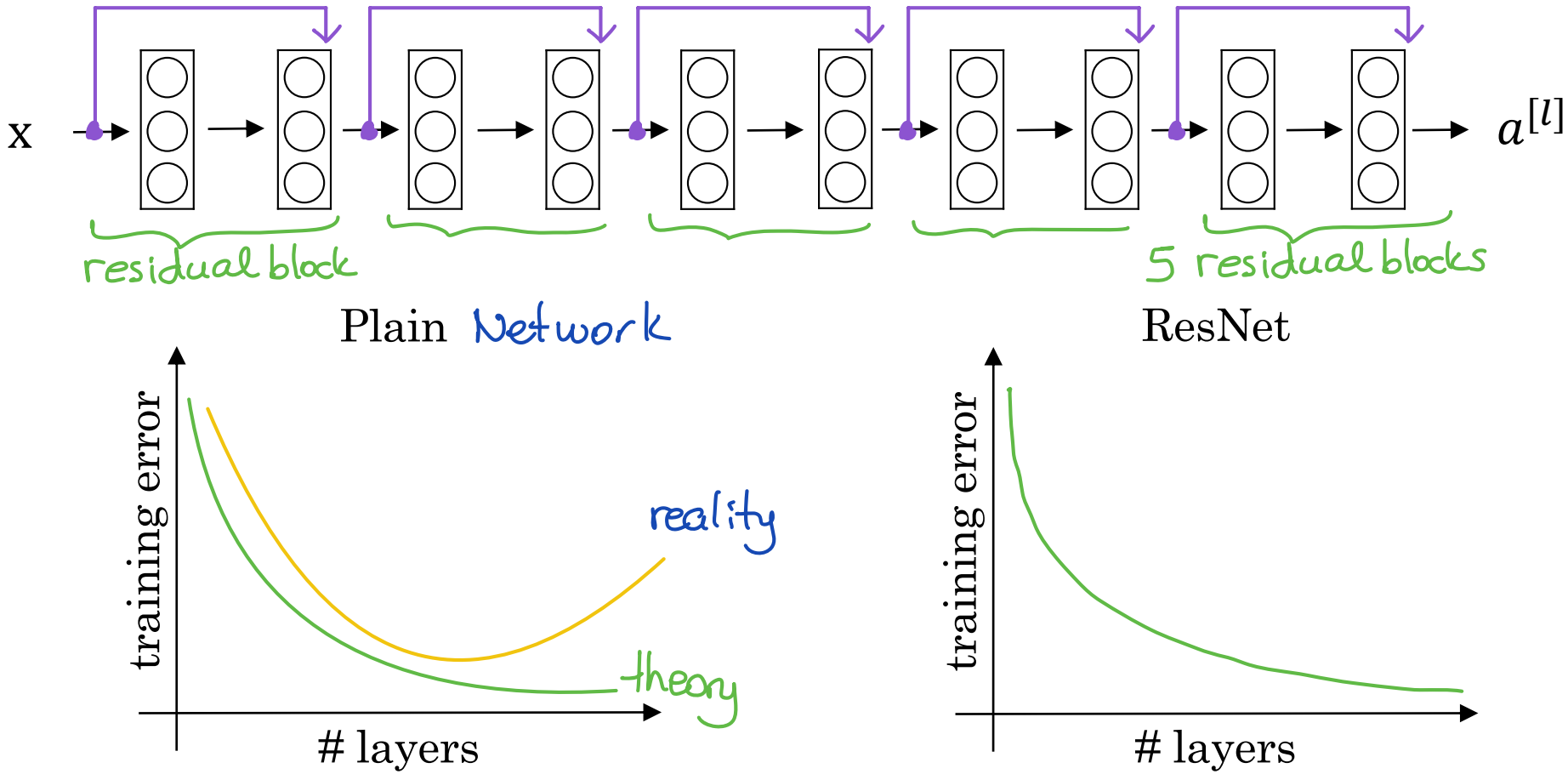
[He et al., 2015. Deep residual networks for image recognition]

Andrew Ng

- Using the skip connection, the information from $a^{[l]}$ follows a short cut to go much deeper into the neural network.
- "Skip connection" skips over almost 2 layers to pass the information $a^{[l]}$ deeper into the network.
- Using residual blocks allows to train much deeper neural networks by helping with the vanishing and exploding gradients.
- ResNet is built by stacking many residual blocks.

- Without "skip connections" the network is called a "plain network".

Residual Network



[He et al., 2015. Deep residual networks for image recognition]

Andrew Ng

deeplearning.ai

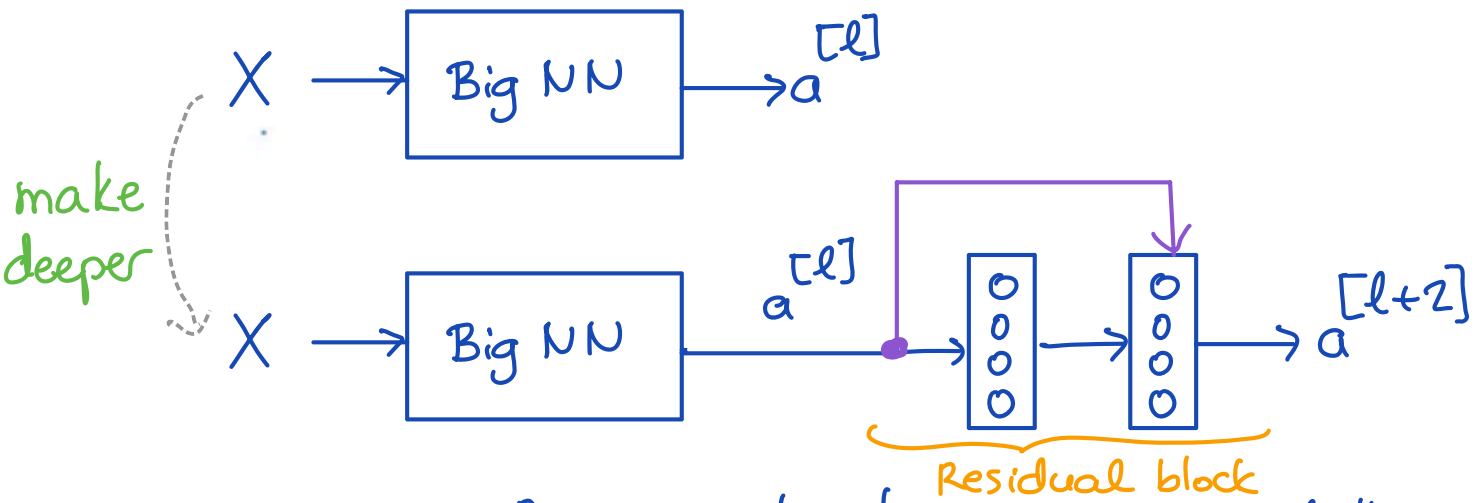
Case Studies

Why ResNets work

- We will go through one example that illustrates why ResNets work so well, at least in the sense that how you can make them deeper and deeper without hurting your ability to at least get them to do well on the training set.
- Doing well on the training set is a pre-requisite to doing well on your hold-out (dev) set or test set.

- If you use a very deep network, it can hurt your ability to train the network to do well on the training set. That's why sometimes you don't want a network that is too deep.
- But this not true when you use a ResNet.

Why do residual networks work?



- We are using ReLU activations, so $a \geq 0$ (all activations are positive)

$$a^{[l+2]} = g(z^{[l+2]} + a^{[l]})$$

$$= g(w^{[l+2]} a^{[l+1]} + b^{[l+2]} + a^{[l]})$$

If $w^{[l+2]} = 0$ and $b^{[l+2]} = 0$

(If we use L_2 regularization $w^{[l+2]}$ and $b^{[l+2]}$ will shrink.)

Andrew Ng

$= g(a^{[l]}) = a^{[l]}$ (since using ReLU activation)

$\Rightarrow$ Identity function is easy for the Residual Block to learn.

$\Rightarrow$ Adding the residual block to the network doesn't hurt the NN's ability to do as well as the simpler network without the extra two layers, since it is easy to learn the identity function (just copy $a^{[l]}$ to $a^{[l+2]}$.

$\Rightarrow$ If the hidden layers learn something more useful than the identity function, the performance may increase

- Another detail: In $g(z^{[l+2]} + a^{[l]})$ we're assuming that the dimensions of $z^{[l+2]}$ and $a^{[l]}$ are the same. If we use "same" convolutions, the dimensions will indeed be the same. and adding the short cut will be possible

- If $a^{[l]}$ is 128 dimensional and $a^{[l+2]}$ is 256 dimensional a matrix W_s is needed to make the dimensions equal:

$$a^{[l+2]} = g(z^{[l+2]} + a^{[l]})$$

$$= g(W^{[l+2]} a^{[l+1]} + b^{[l+2]} + W_s a^{[l]})$$

$a^{[l+2]} \in \mathbb{R}^{256}$, $W_s \in \mathbb{R}^{256 \times 128}$, $a^{[l]} \in \mathbb{R}^{128}$

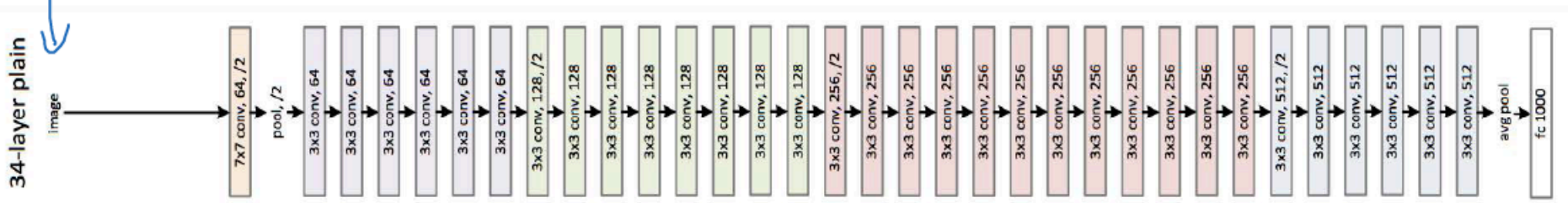
- W_s could be a matrix with learnable parameters. OR
- W_s could be a fixed matrix that just implements zero padding that takes $a^{[l]}$ and zero pads to 256 dimensions.

ResNet

input image

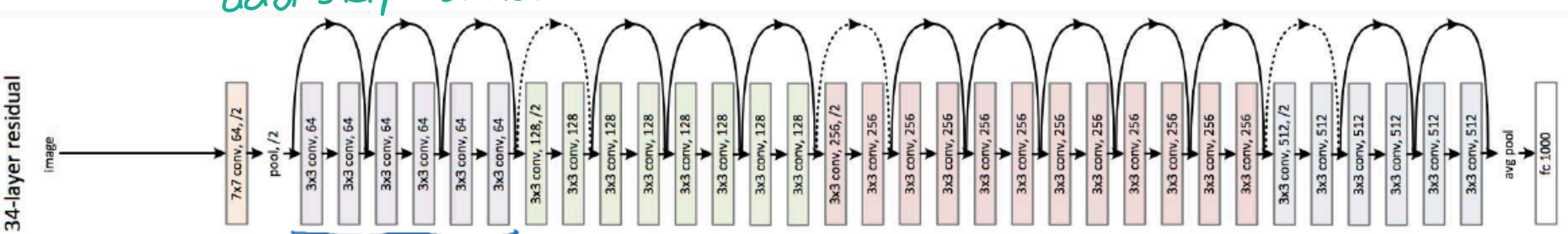
1000 classes

Plain



add skip connections

ResNet



$z^{[l+2]} + a^{[l]}$

3x3 same convolutions

Pool

Pool → dimensions change

W_s

need this adjustment

[He et al., 2015. Deep residual networks for image recognition]

Andrew Ng

dimensions are preserved: $z^{[l+2]} + a^{[l]}$ makes sense

deeplearning.ai

Case Studies

Network in Network and 1×1 convolutions

Why does a 1 × 1 convolution do?

1	2	3	6	5	8
3	5	5	1	3	4
2	1	3	4	9	3
4	7	8	5	7	9
1	5	3	7	4	8
5	4	9	8	3	5

$6 \times 6 \times 1$

*

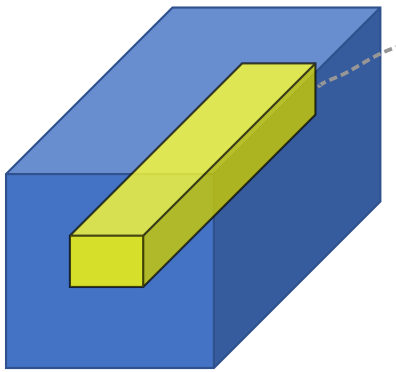
1x1 filter

2

=

image is multiplied by 2

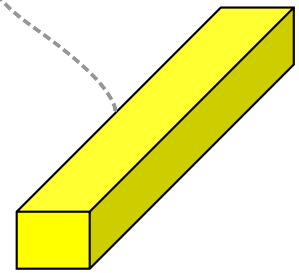
2	4	6	·	·	·



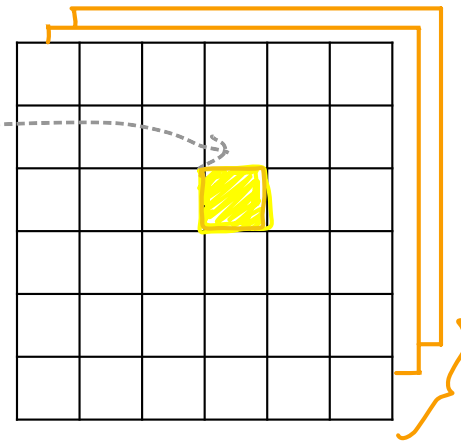
$6 \times 6 \times 32$

multiply these "arrays" and add, gives a real number + apply ReLU

*



=



#filters

$1 \times 1 \times 32$

size of the filter

(# filters of them: $n_c^{[l+1]}$)

$6 \times 6 \times$ # filters

[Lin et al., 2013. Network in network]

(There are 36 different positions for the 1x1 filter)

Activations represent some interchannel relationship.

Andrew Ng

- If we convolve a 6x6x1 image with a 1x1 filter (of value=2), the input image will be multiplied by 2.
- 1x1 convolution is not useful if your input image has only 1 channel.
- But if the #channels in the input image is larger than 1, 1x1 convolution may do something meaningful by combining the information in the input channels.
- 1x1 convolution was used to reduce the number of channels while introducing non-linearity. This will reduce the # of computations.
- If you convolve a 6x6x32 input with a 1x1x32 filter the output size will be 6x6x1 (e.g. 32 channels may be "averaged" to a single channel.

Why does a 1 × 1 convolution do?

image is multiplied by 2

1	2	3	6	5	8
3	5	5	1	3	4
2	1	3	4	9	3
4	7	8	5	7	9
1	5	3	7	4	8
5	4	9	8	3	5

6 × 6 × 1

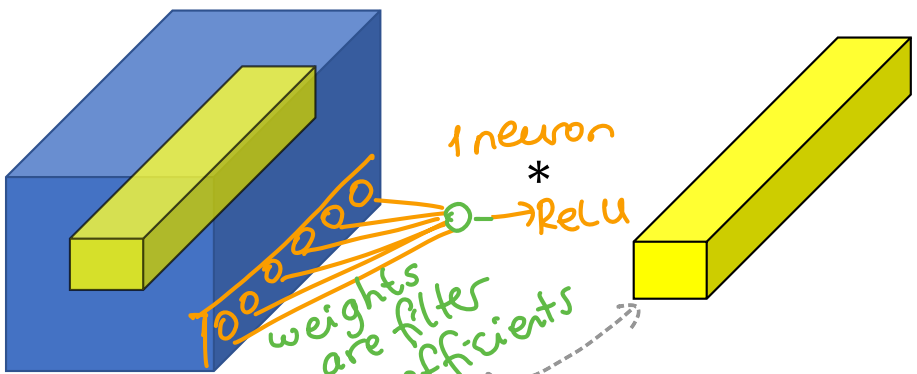
*

1×1 filter

2

=

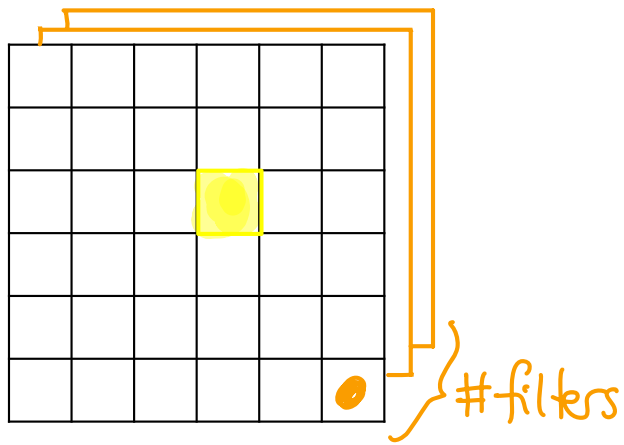
2	4	6	·	·	·



6 × 6 × 32

=

1 × 1 × 32



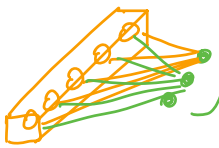
6 × 6 × # filters

[Lin et al., 2013. Network in network]

Andrew Ng

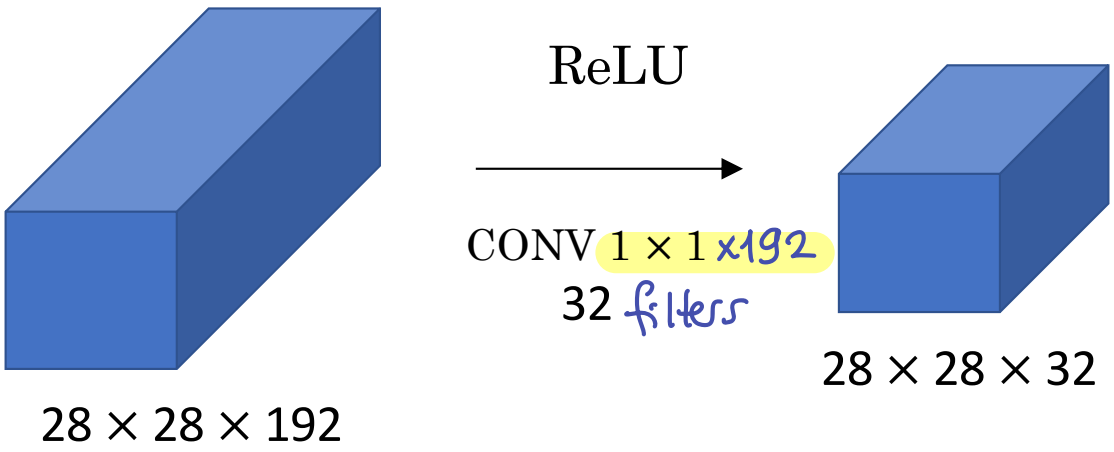
- 1×1×32 filter is equivalent to as if you have 1 neuron and you multiply 1 slice through the channels and add them up. Then pass thru ReLU.

- If you have more 1x1 filters it will be as if you have a fully connected layer:



$\Rightarrow$ called as Network in Network

Using 1×1 convolutions



- If you want to shrink the height (n_H) and width (n_W), you can use a pooling layer.
- If you want to shrink the # channels (n_c) use a 1x1 convolution
- If you don't decreare n_c, 1x1 conv just addr nonlinearity and allows you to learn a more complex function.

[Lin et al., 2013. Network in network]

Andrew Ng

deeplearning.ai

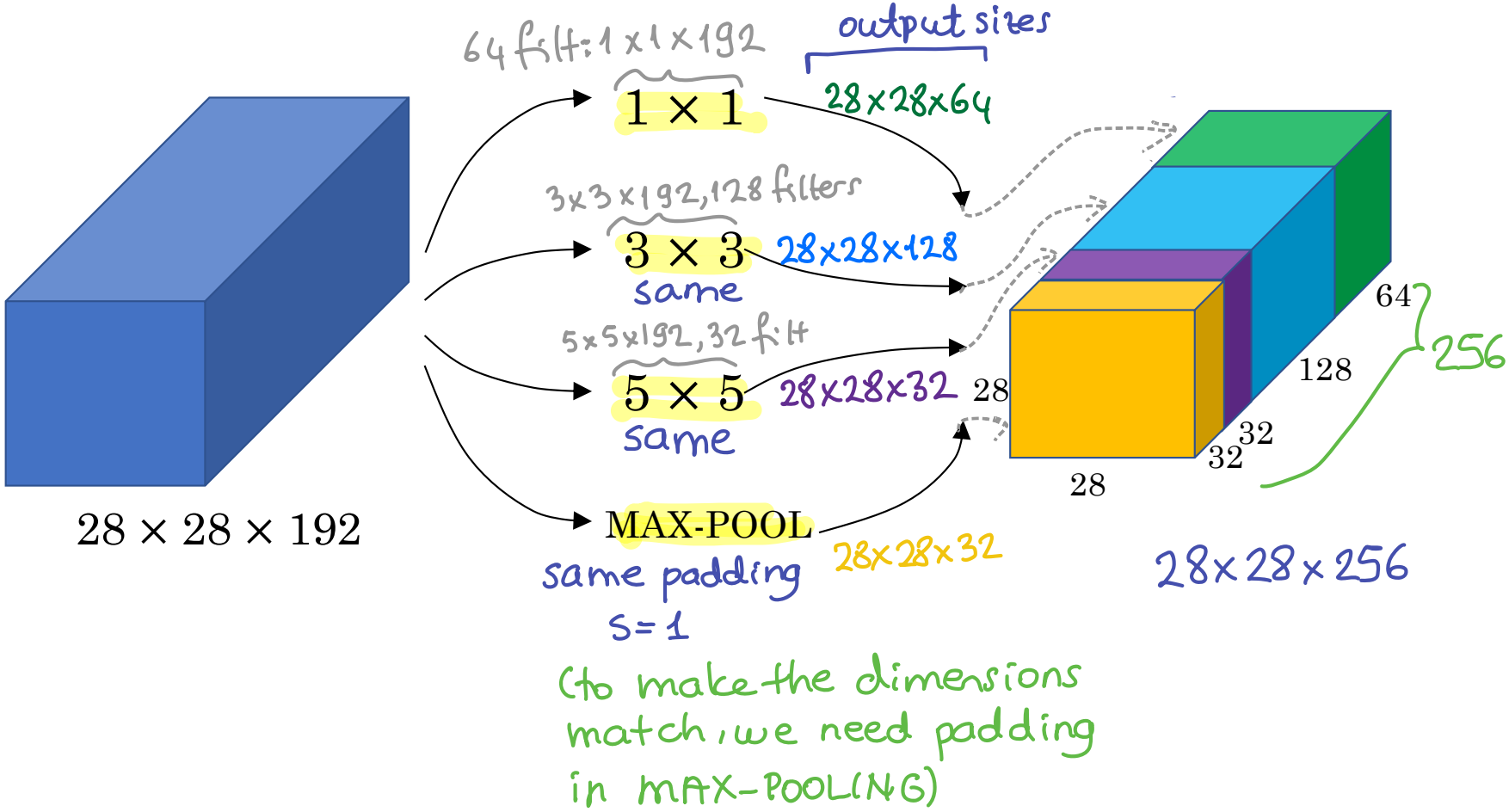
Case Studies

Inception network motivation

- When designing a layer of ConvNet, you need to decide if you want a 1x3 filter, or a 3x3 filter, or 5x5, or do you want a pooling layer?
- Inception layer idea: why not do them all!
- This makes the network architecture more complicated, but it also works remarkably well.

(inception: act, process or instance of beginning: commencement)
(dictionary meaning)

Motivation for inception network



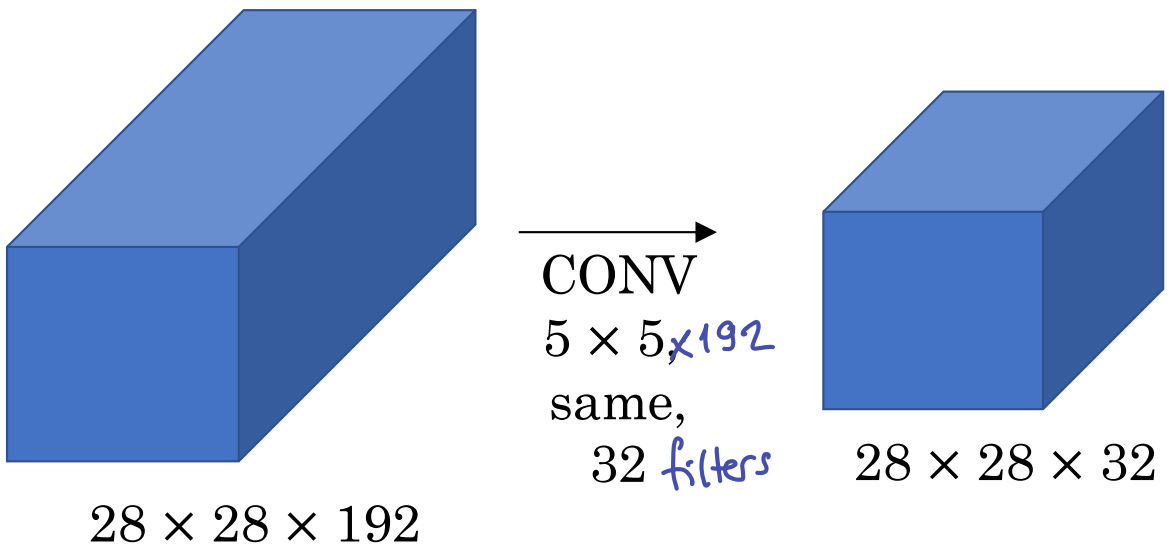
(to make the dimensions match, we need padding in MAX-POOL(NG))

[Szegedy et al. 2014. Going deeper with convolutions]

IDEA: Instead of picking one of these filter sizes or pooling, you can do them all and just concatenate all the outputs, and let the network learn whatever parameters it wants to use, whatever combination of these filter sizes it wants.

- There's a problem with the inception layer as described above: computational cost.

The problem of computational cost



Computational cost: 32 filters. Filters are 5x5x192

Need to compute: 28x28x32 numbers

For each number we need: 5x5x192 multiplications

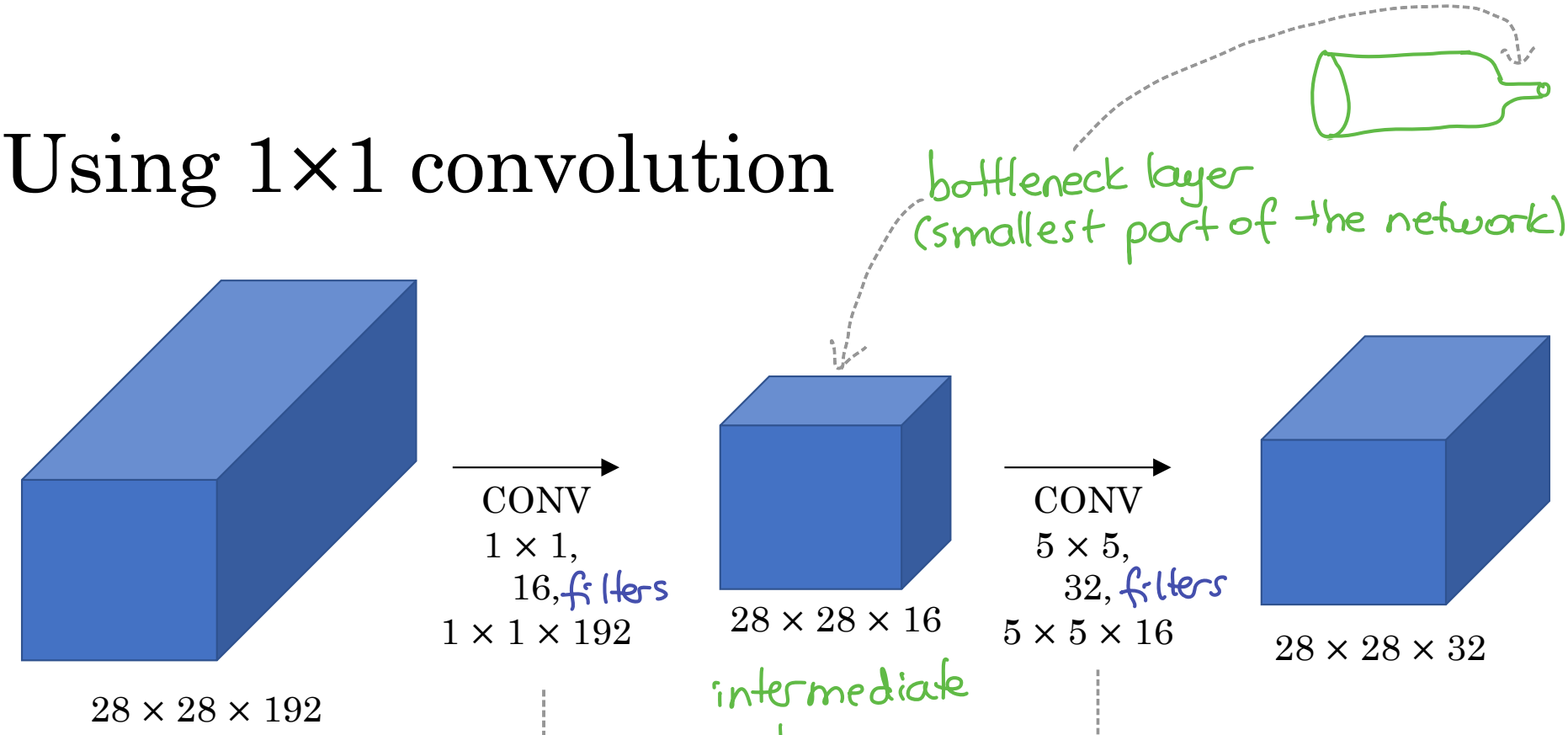
28x28x32 x 5x5x192 = 120M multiplications

pretty expensive operation

Goal: Reduce this by 1/10 by using 1x1 convolutions.

Andrew Ng

Using 1×1 convolution



Computational Cost:

28×28×16 × 192 = 2.4M
#outputs filter size

28×28×32 × 5×5×16 = 10M
#outputs filter size

\+

12.4M total multiplications

Previous slide: 120M multiplications ↘ one-tenth

(#additions ≅ #multiplications)

Andrew Ng

- Summary: If you're building a layer of a NN and you don't want to have to decide, do you want a 1x1, 3x3, 5x5, or pooling layer, the inception module let's you do them all and concatenates the results.
  - To reduce the computational cost, we use 1x1 convolution to create a bottleneck layer.

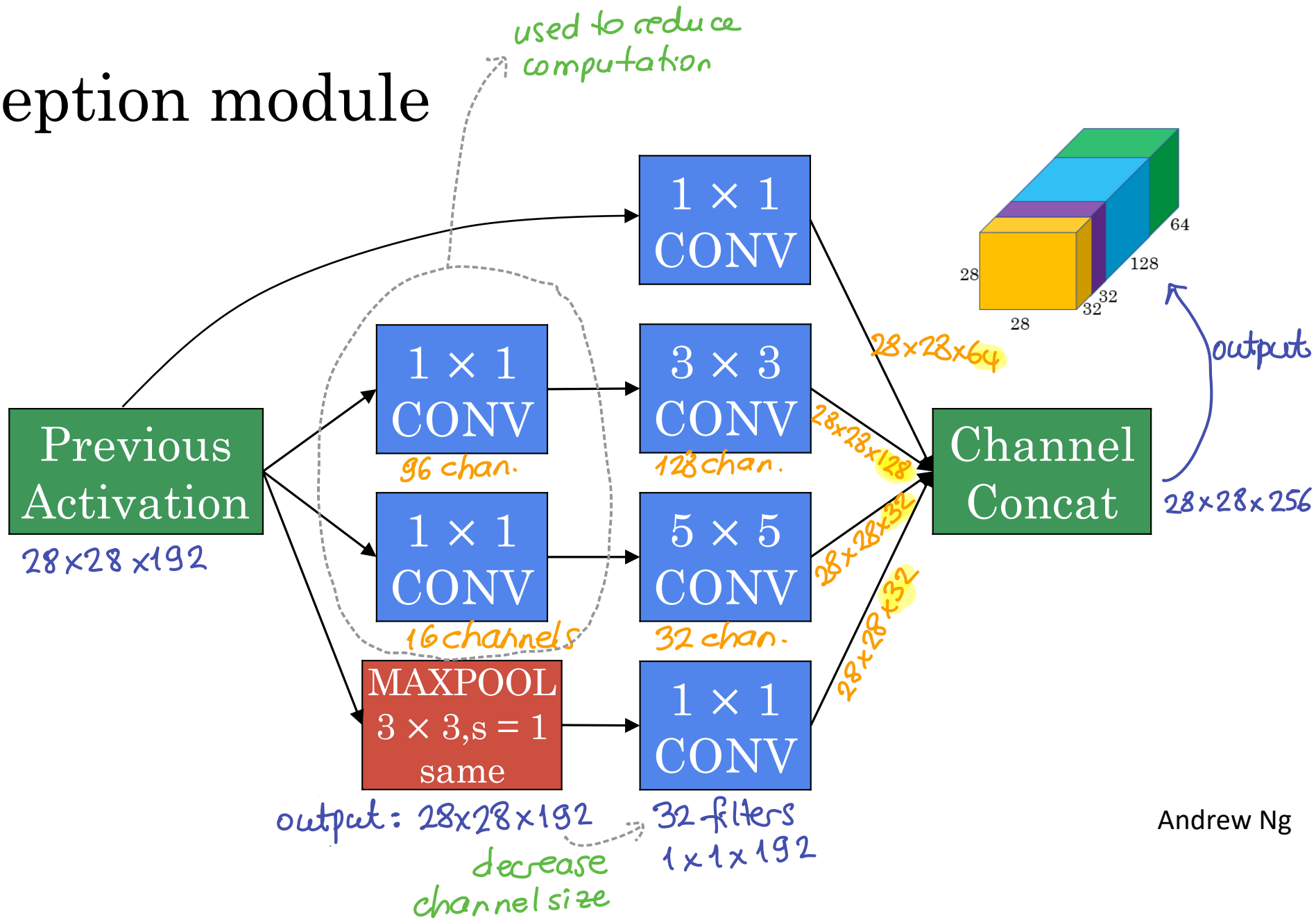
- Does shrinking down the representation size so dramatically hurt the performance of the neural network?
  - So long as you implement the bottleneck layer within reason, you can shrink down the representation size significantly, and it doesn't seem to hurt the performance of the network, but saves you a lot of computation.

deeplearning.ai

Case Studies

Inception network

Inception module

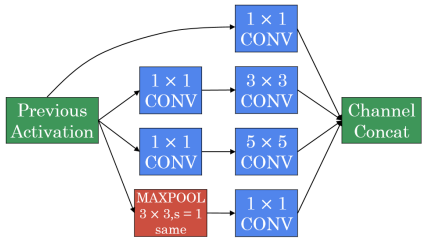


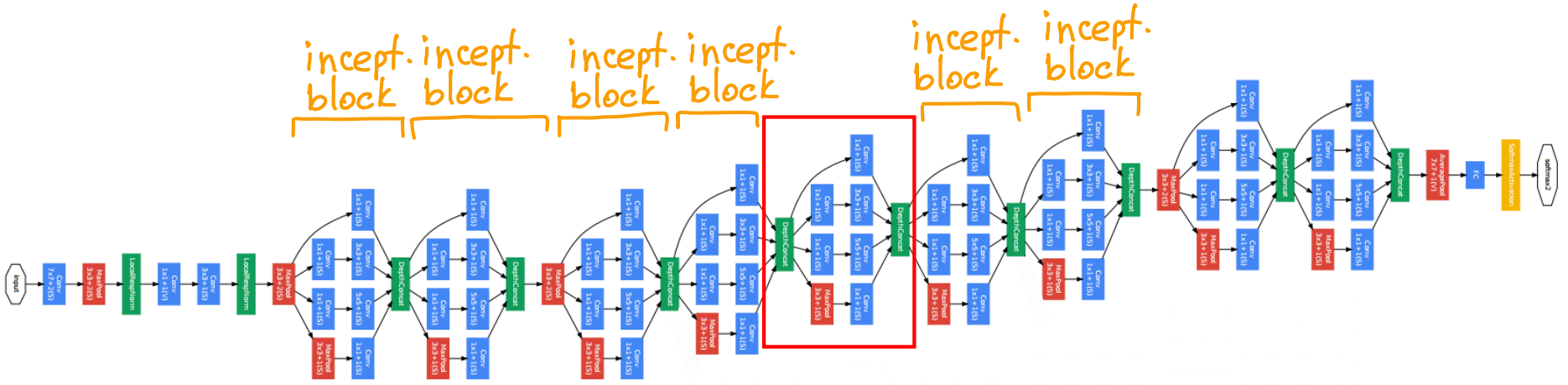
Andrew Ng

Inception network

Puts many inception modules together.

Inception Module: (Block)



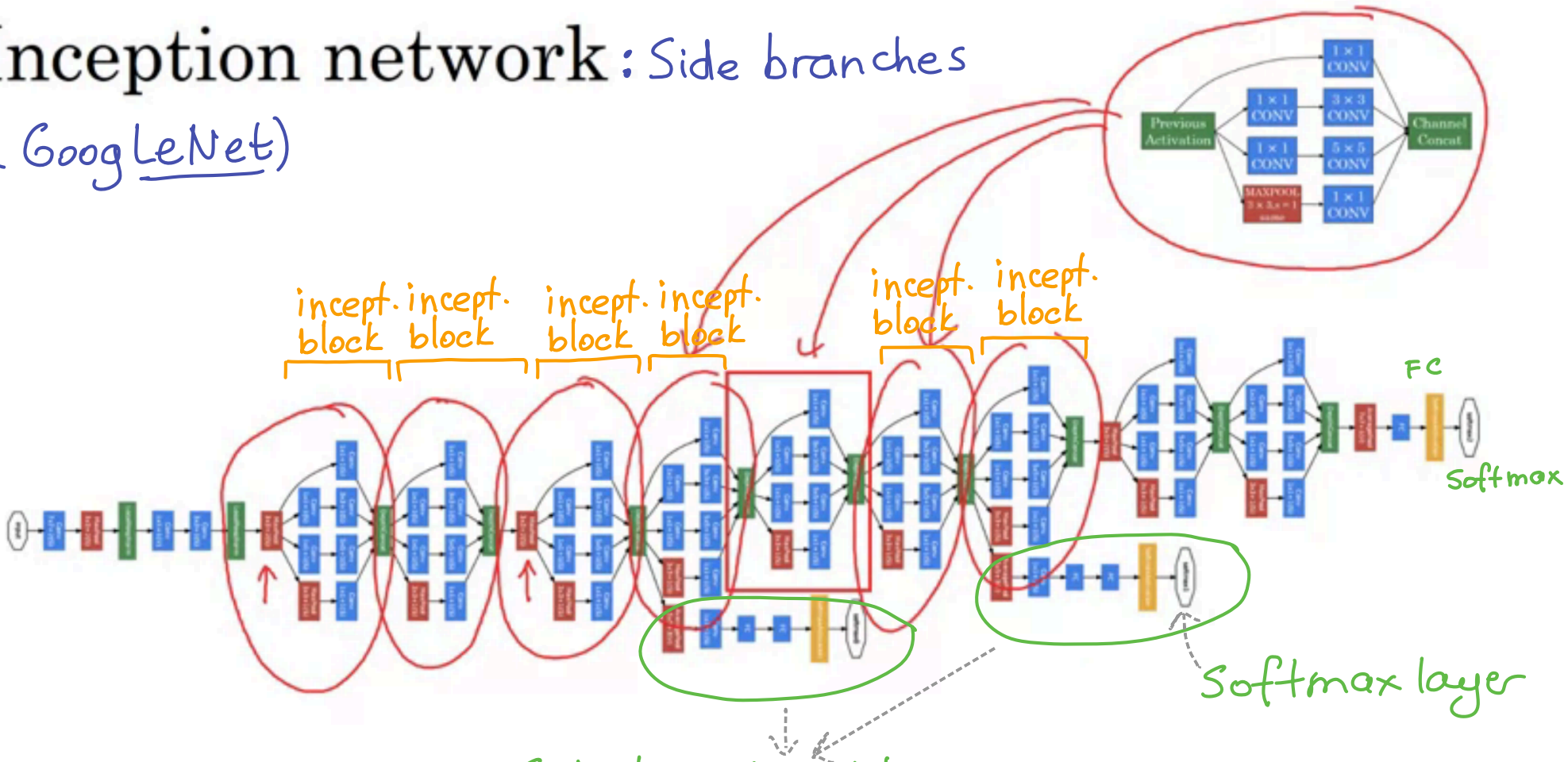


[Szegedy et al., 2014, Going Deeper with Convolutions]

Inception Module:

Inception network: Side branches

(GoogLeNet)



Side branches take a hidden layer and tries to use that to make a prediction.

[Szegedy et al., 2014, Going Deeper with Convolutions]

Idea of side branches: To ensure that the features computed even at intermediate layers aren't too bad for predicting the output class of the image.

- This has a regularizing effect on the inception network and helps to prevent overfitting.

- Other versions exist in the literature: inception v2, inception v3, inception v4. It is also combined with the ResNet idea of skip connections, that sometimes even works better.

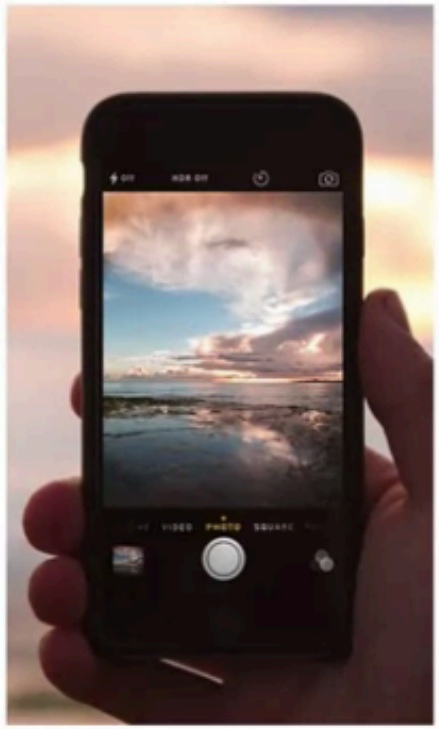
deeplearning.ai

Convolutional Neural Networks

MobileNet

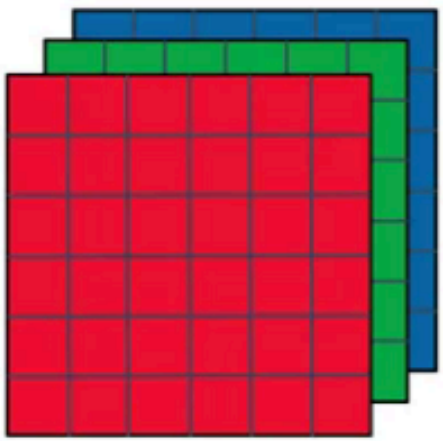
Motivation for MobileNets

- Low computational cost at deployment
- Useful for mobile and embedded vision applications
- Key idea: Normal vs. depthwise-separable convolutions



[Howard et al. 2017, MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications]

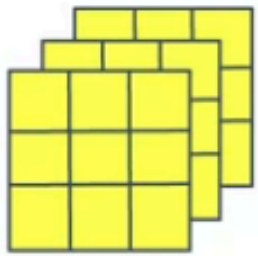
Normal Convolution



6 x 6 x 3

$n \times n \times n_c$

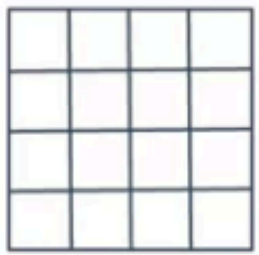
*



3 x 3 x 3

$f \times f \times n_c$

=

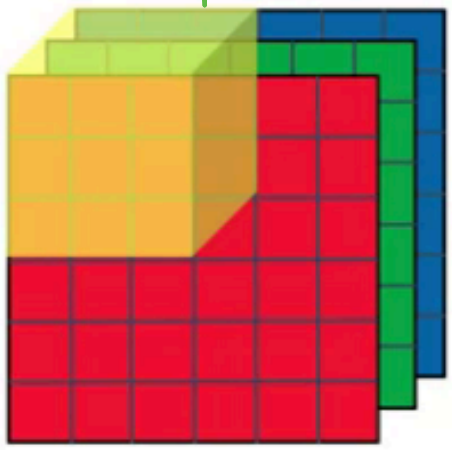


4 x 4

Normal Convolution

27 multiplications

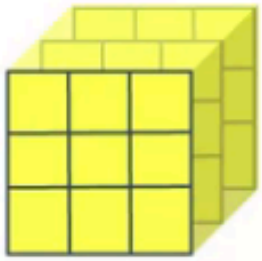
compute output here



6 x 6 x 3

$n \times n \times n_c$

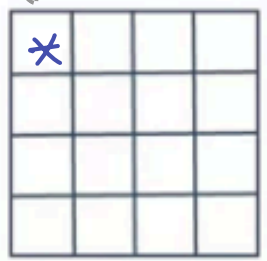
*



3 x 3 x 3

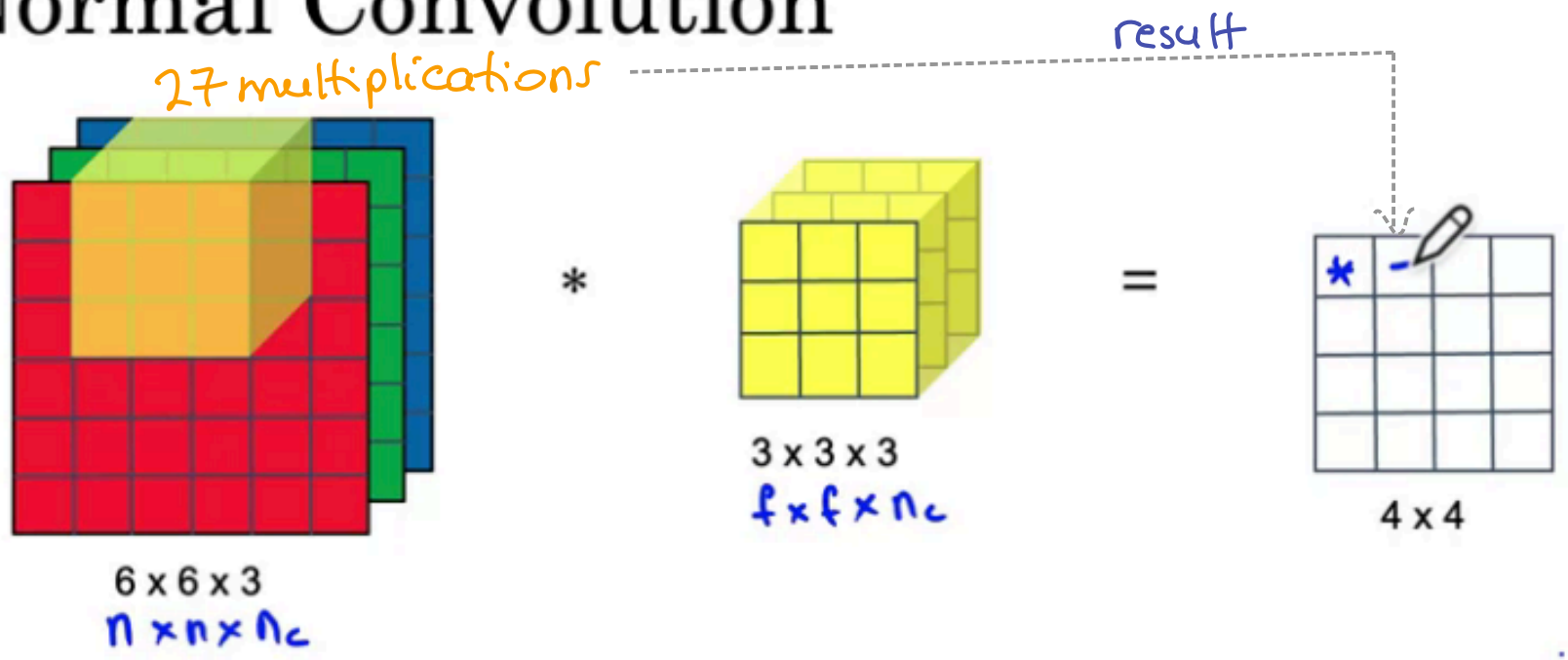
$f \times f \times n_c$

=

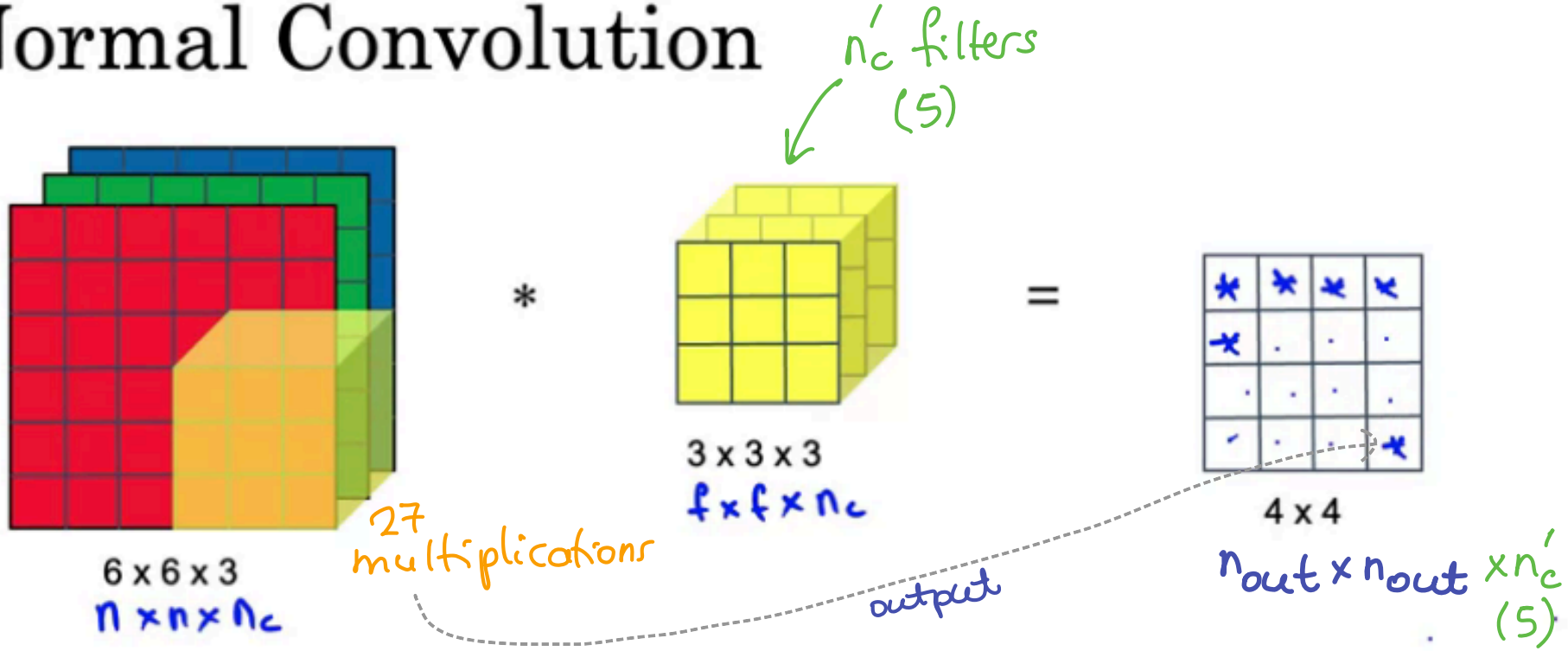


4 x 4

Normal Convolution



Normal Convolution

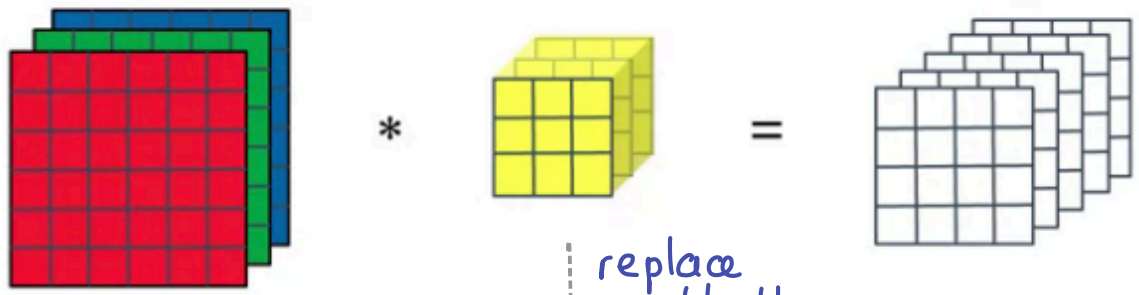


Computational cost = # filter params × # filter positions × # of filters

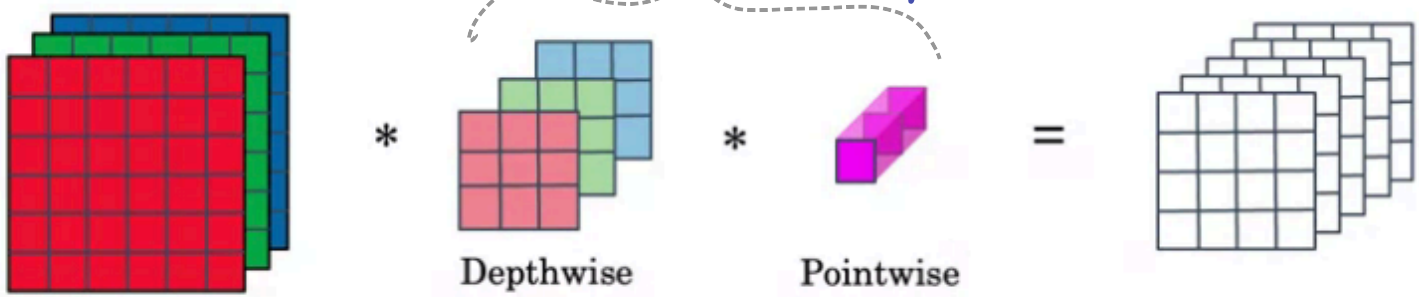
2160 = 3×3×3 × 4×4 × 5

Depthwise Separable Convolution

Normal Convolution



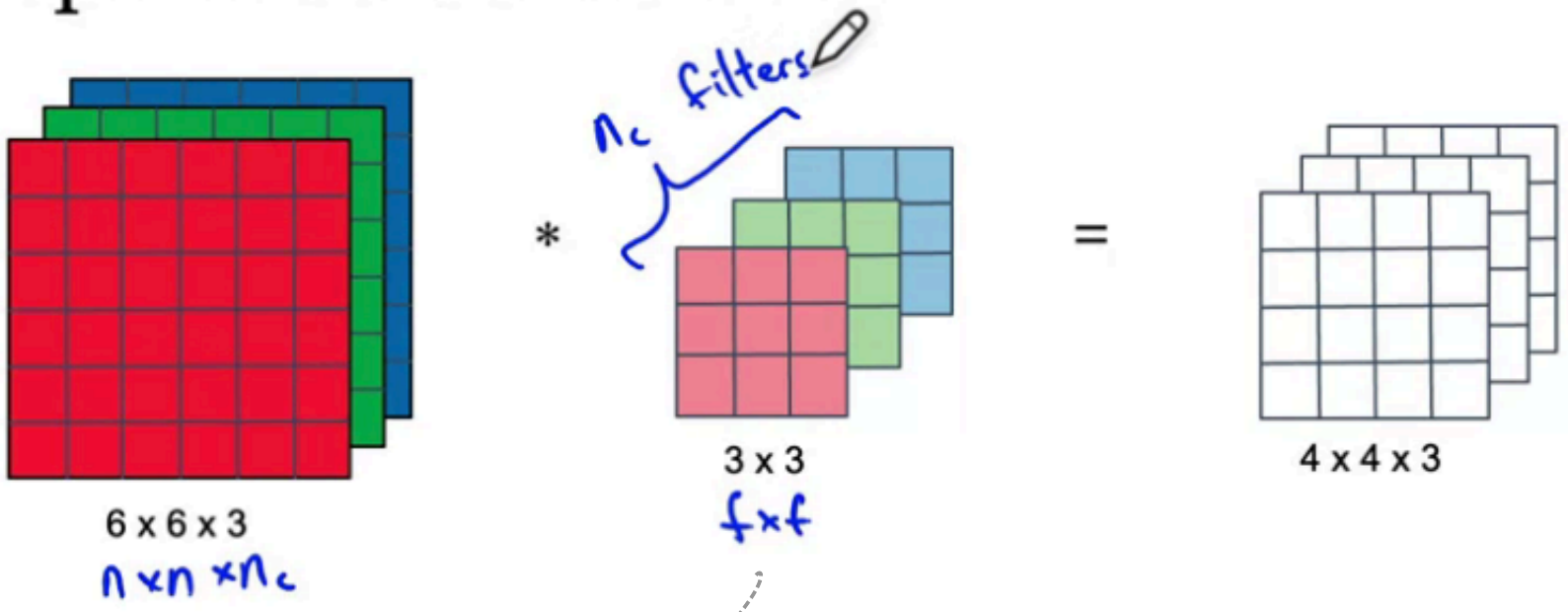
Depthwise Separable Convolution



Depthwise separable convolution

Andrew Ng

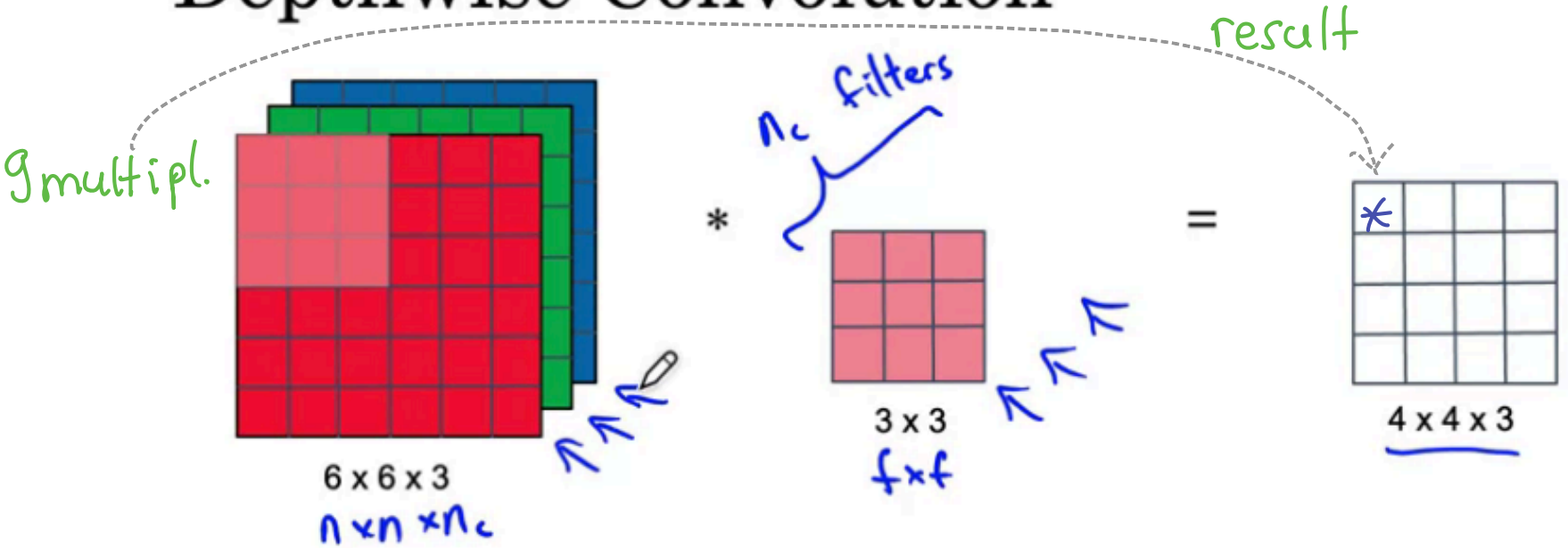
Depthwise Convolution



apply one of these filters to
one of each of the input channels

Andrew Ng

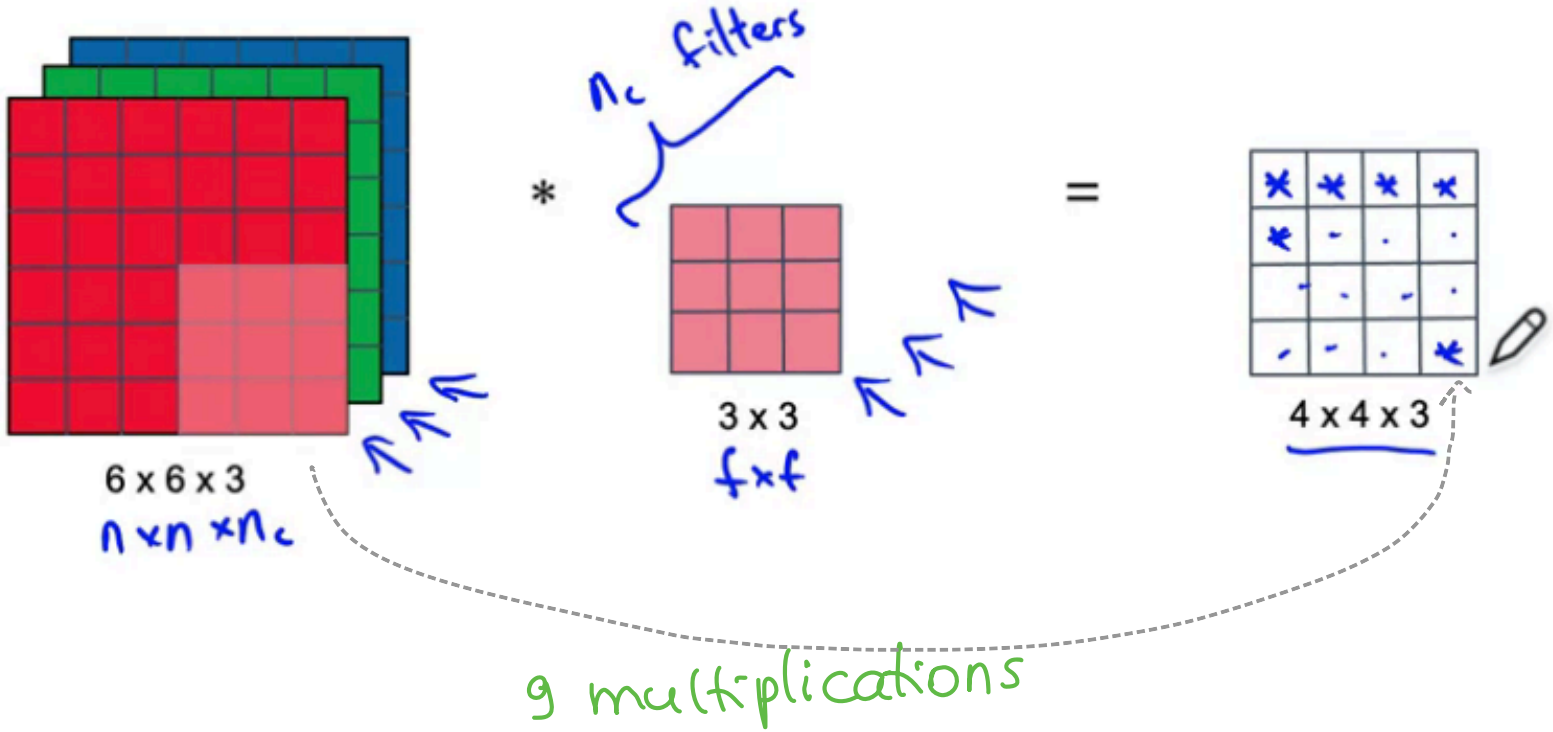
Depthwise Convolution



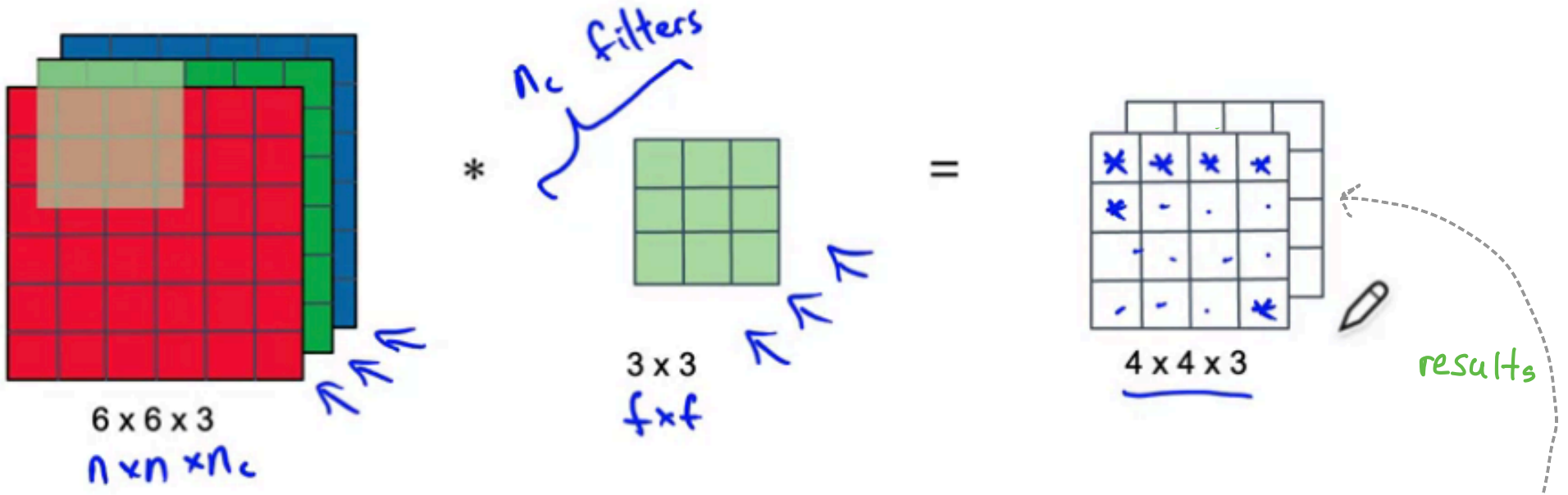
- Convolve the red channel (6x6) with the red filter (3x3) to obtain the 4x4 output.

Andrew Ng

Depthwise Convolution

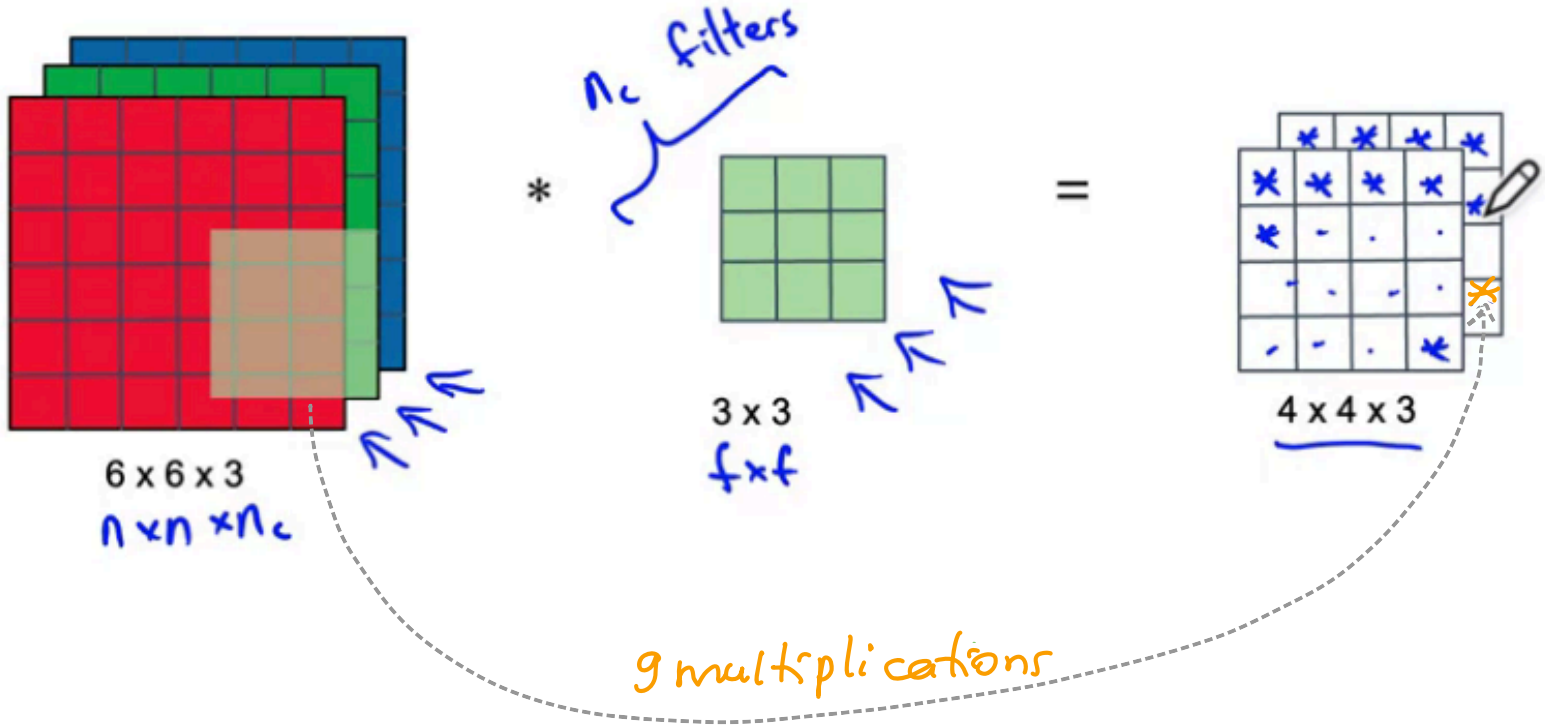


Depthwise Convolution

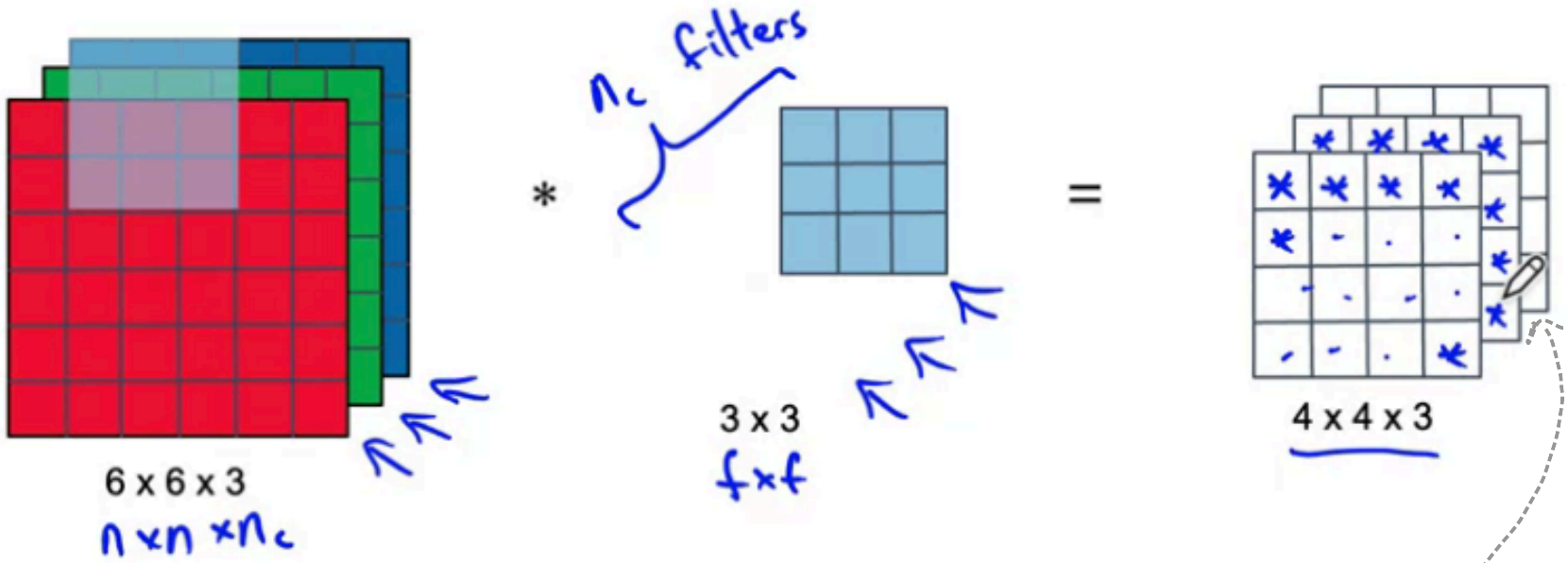


- Convolve the green channel of the input (6x6) by the green filter and obtain the 4x4 result shown.

Depthwise Convolution

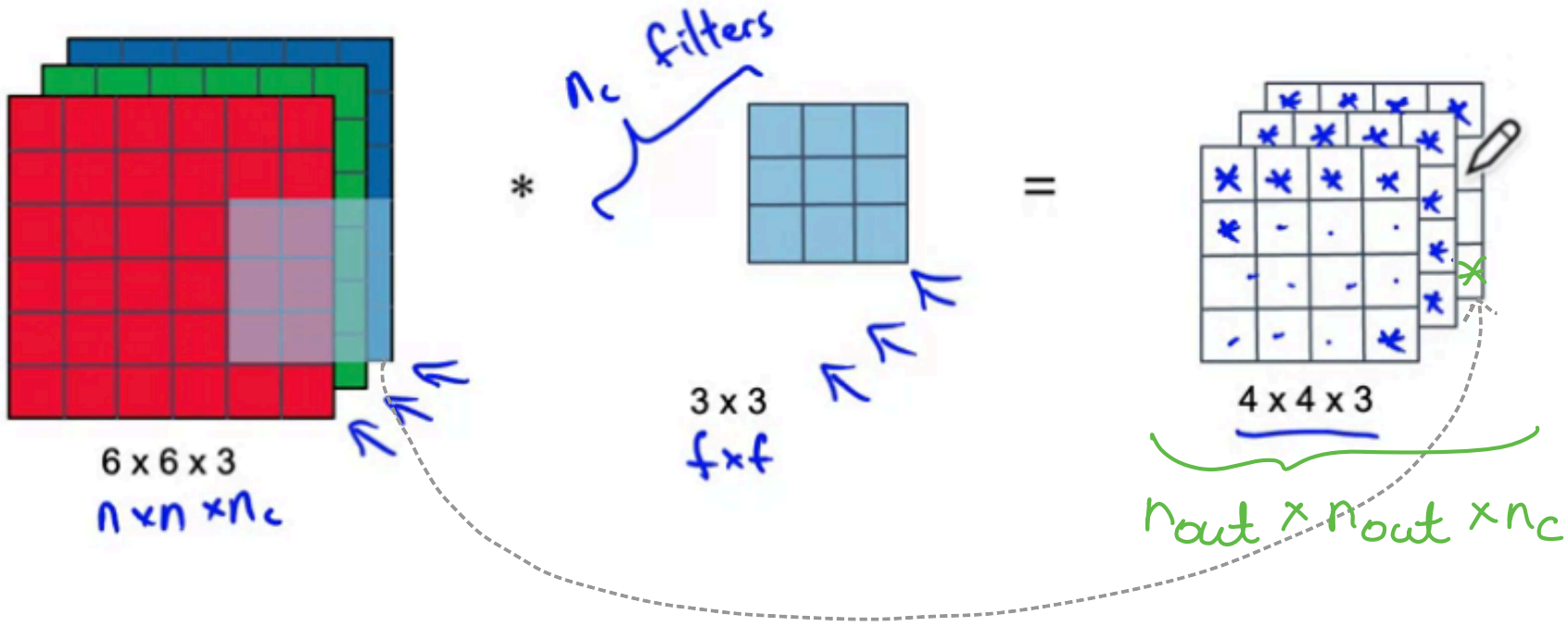


Depthwise Convolution

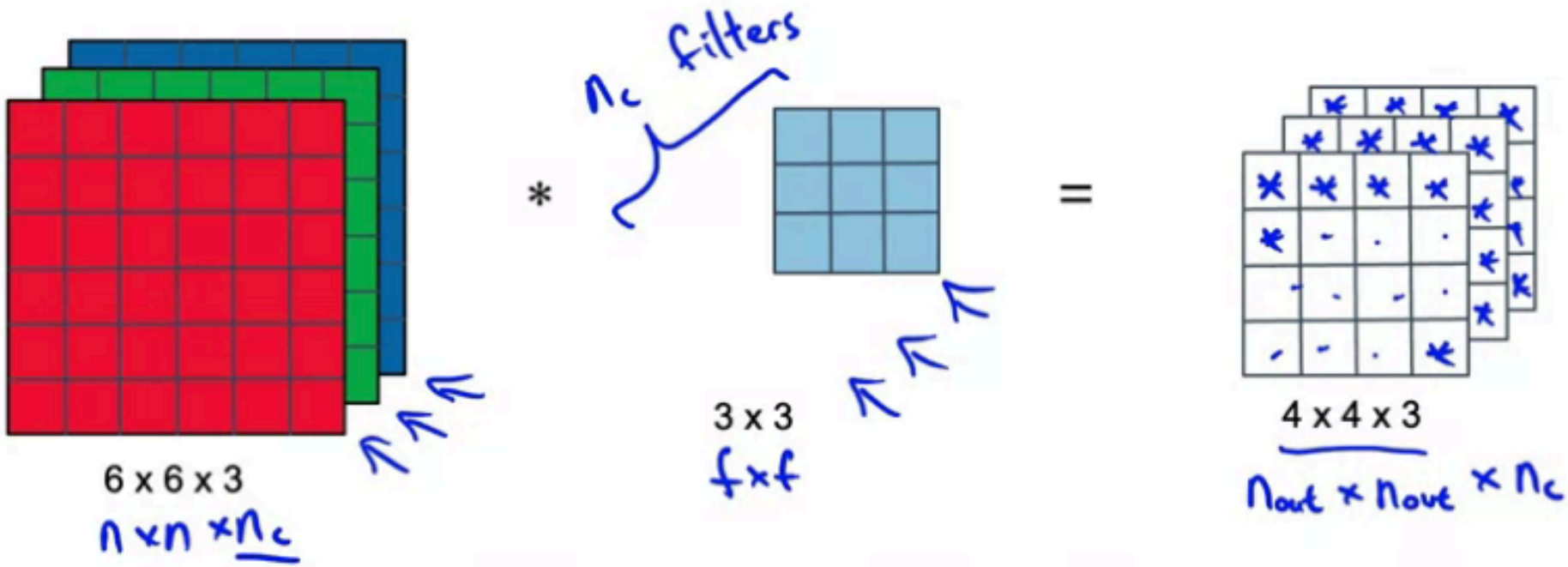


* Convolve the blue channel of the input with the blue filter, and obtain the 4x4 result.

Depthwise Convolution



Depthwise Convolution

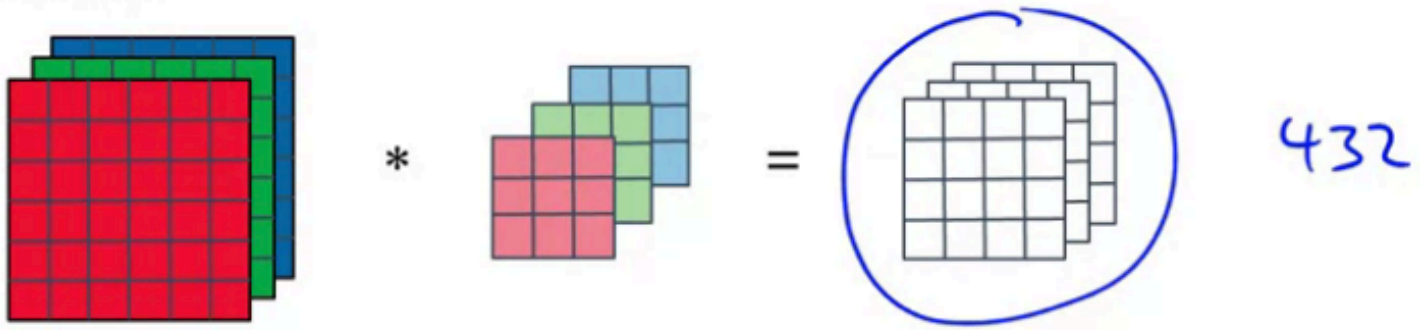


Computational cost	=	#filter params	x	# filter positions	x	# of filters
432	=	3x3	x	4x4	x	3

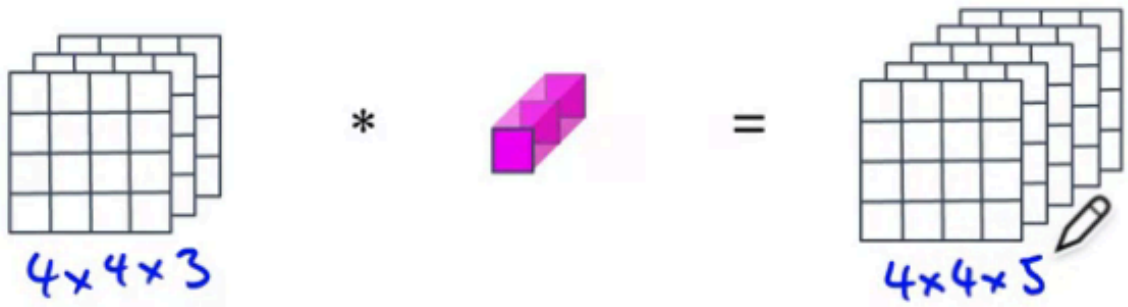
multiplications

Depthwise Separable Convolution

Depthwise Convolution



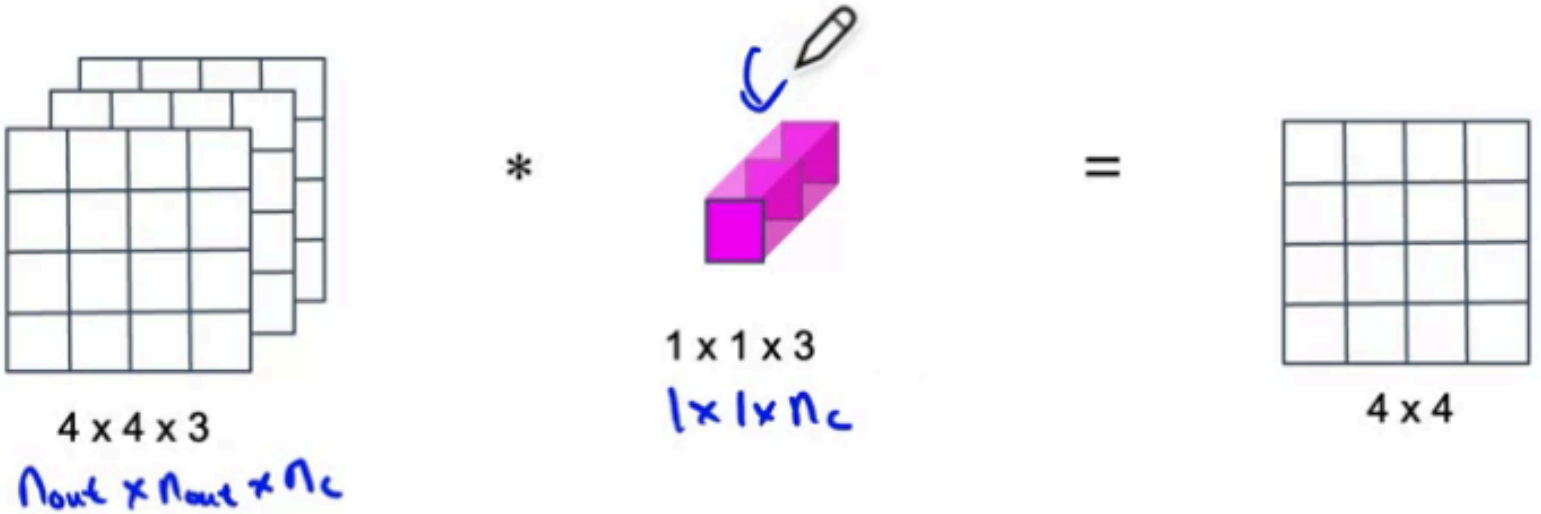
Pointwise Convolution



Next Step: Carry out pointwise convolution which will convert a 4x4x3 to 4x4x5 output.

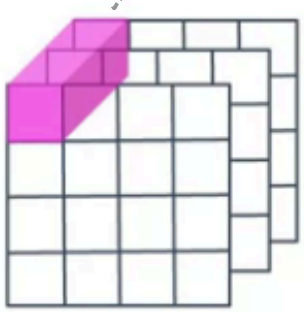
Andrew Ng

Pointwise Convolution



Pointwise Convolution

3 multiplications, add them up



4 x 4 x 3

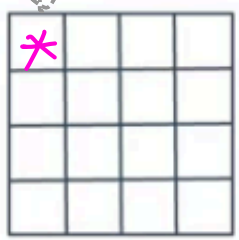
$n_{out} \times n_{out} \times n_c$

*

1 x 1 x 3

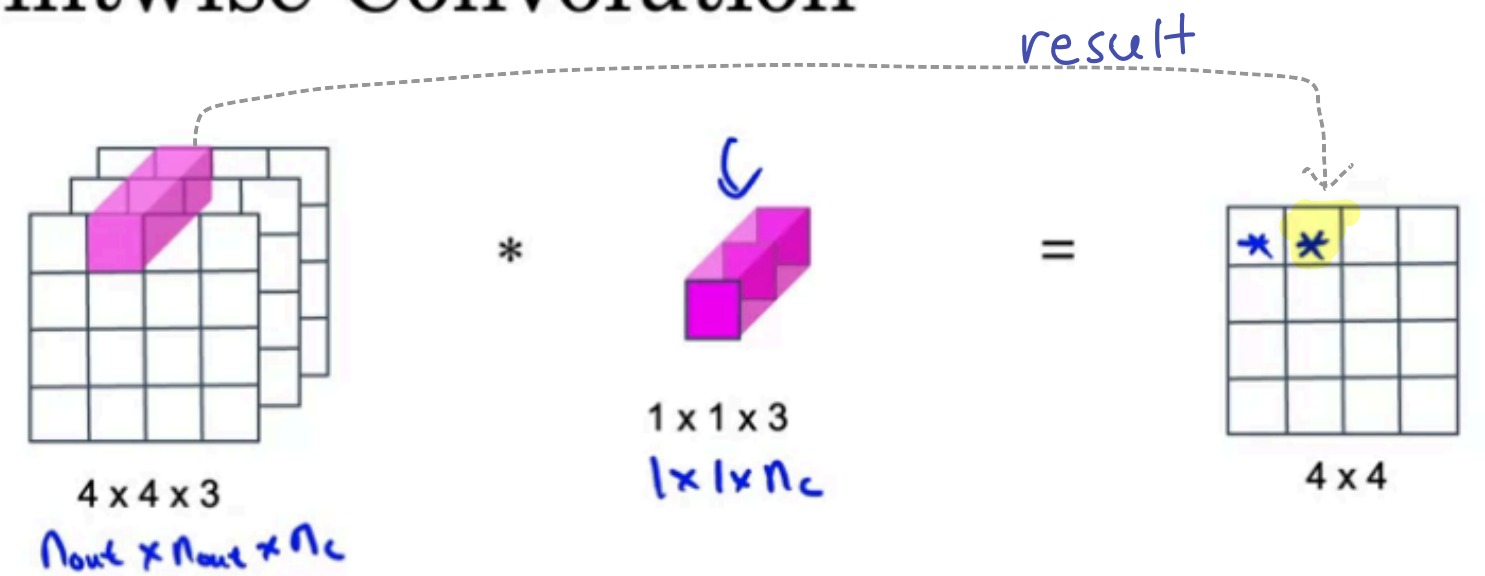
$1 \times 1 \times n_c$

=



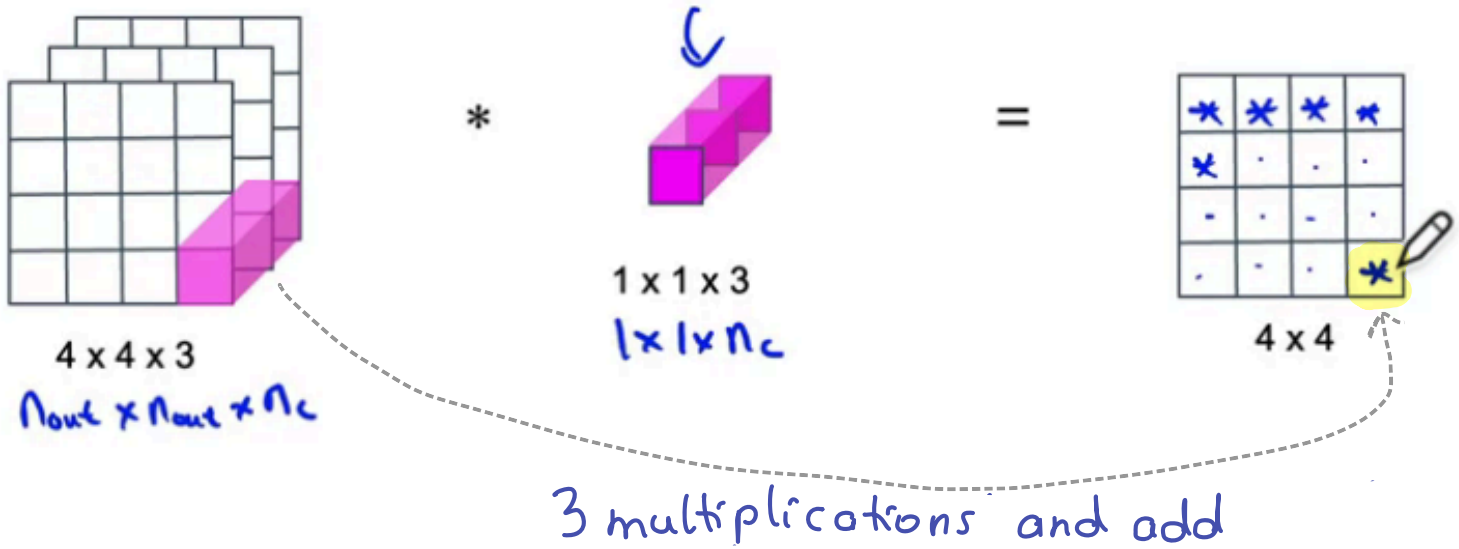
4 x 4

Pointwise Convolution



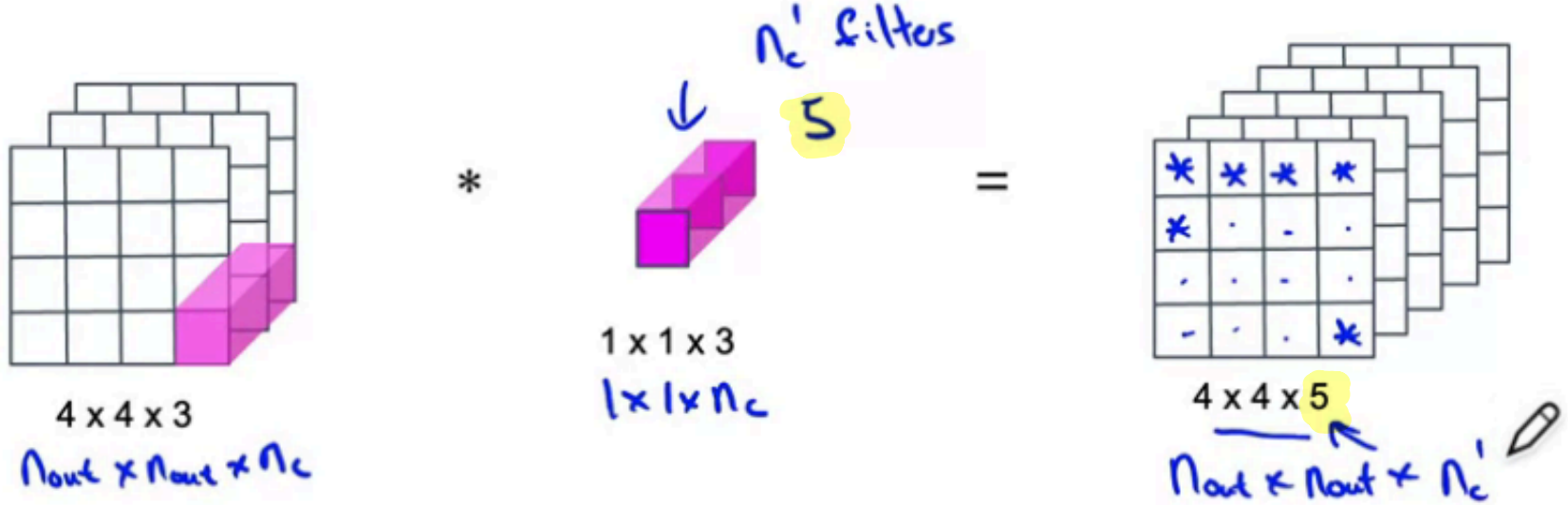
Andrew Ng

Pointwise Convolution



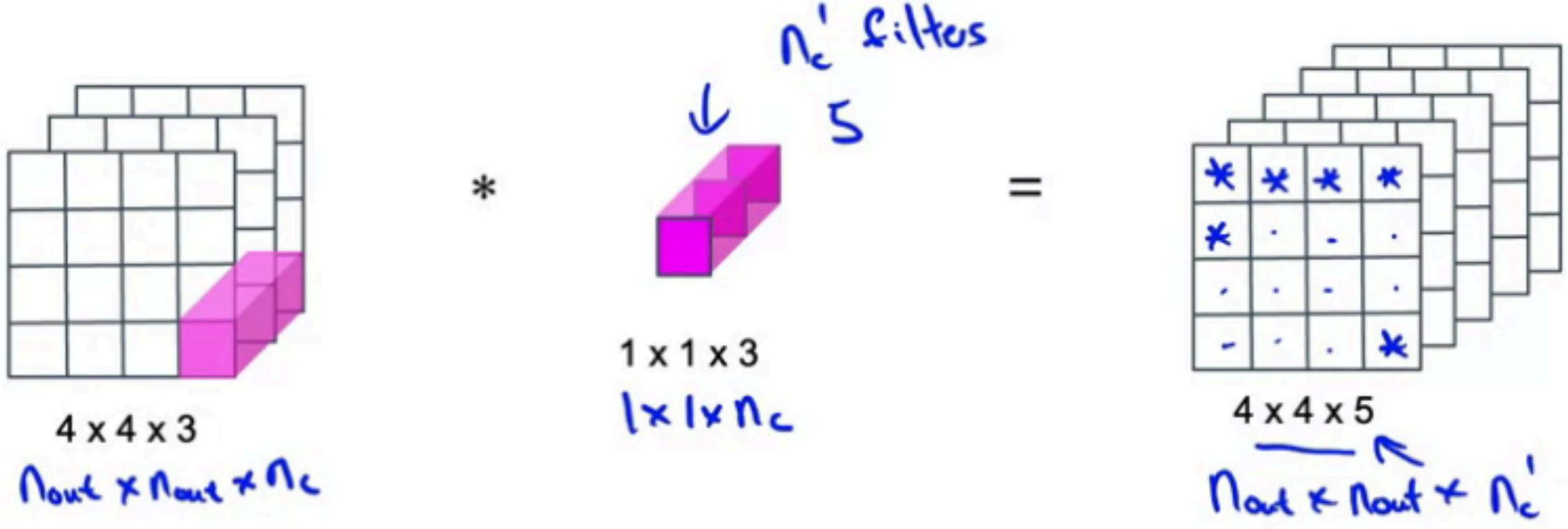
Andrew Ng

Pointwise Convolution



Andrew Ng

Pointwise Convolution



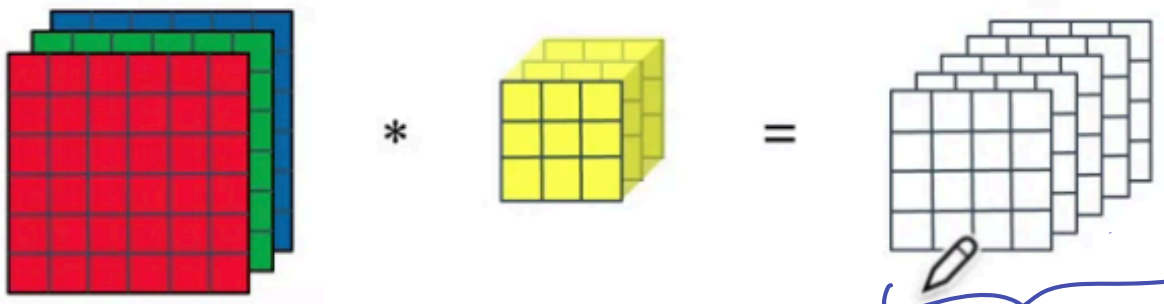
Computational cost = #filter params x # filter positions x # of filters

1x1x3 x 4x4 x 5

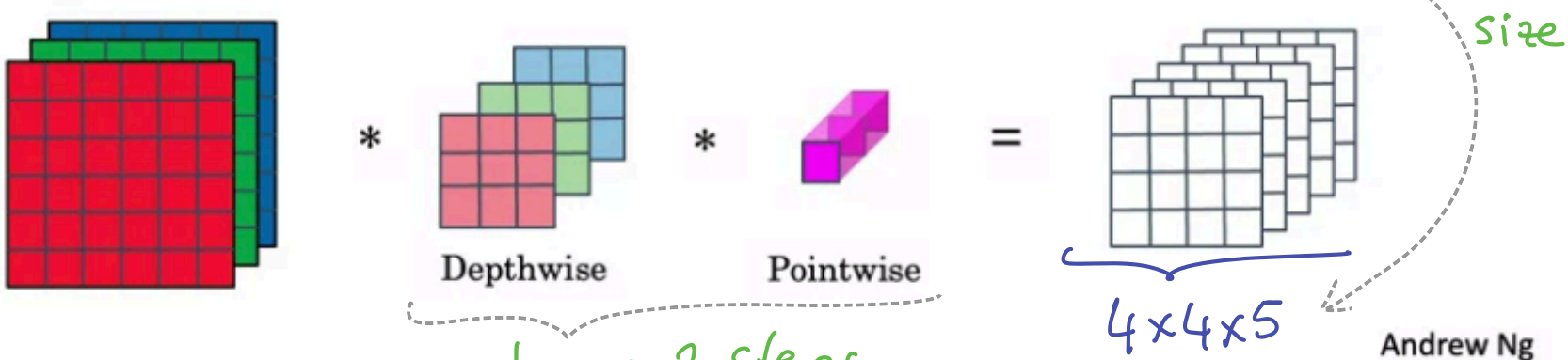
240 = multiplications

Depthwise Separable Convolution

Normal Convolution



Depthwise Separable Convolution



Cost Summary

Cost of normal convolution 2160 (# multiplications)

Cost of depthwise separable convolution

depthwise + pointwise

432 + 240 = 672 (# multiplications)

$\frac{672}{2160} = 0.31$

(The depthwise separable convolution was about 31% as computationally expensive as the normal convolution.)

[Howard et al. 2017, MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications]

- In the above paper, it is shown that in the general case, the ratio of the cost of the depthwise separable convolution compared to the normal convolution is:

$$\frac{1}{n_c'} + \frac{1}{f^2} = \frac{1}{5} + \frac{1}{9} \cong 0.31$$

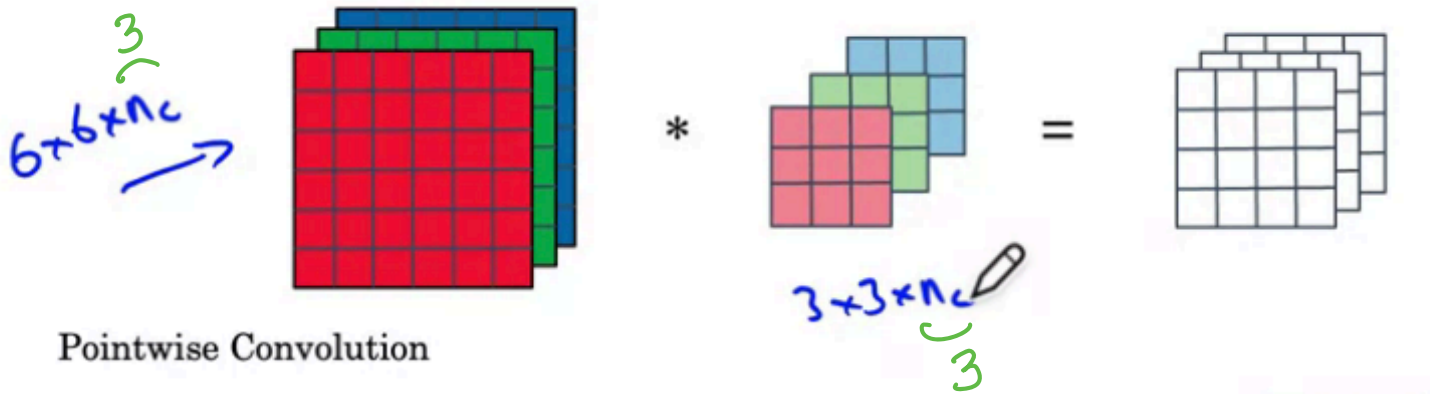
in a typical network: $n_c' \cong 512$, $f=3$ (typical parameters)

$$\Rightarrow \text{ratio} = \frac{1}{512} + \frac{1}{3^2} \cong \frac{1}{9} \quad \sim 10 \text{ times cheaper}$$

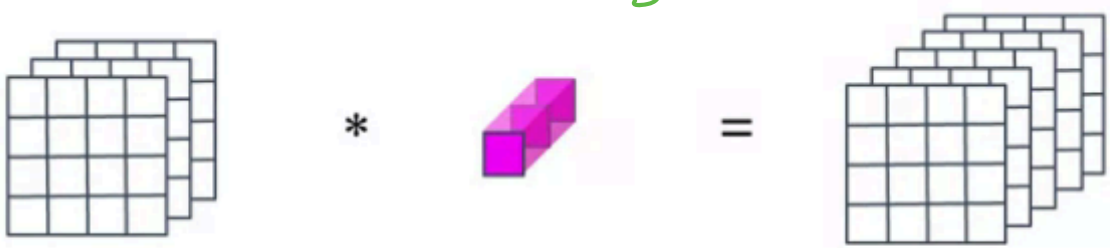
- Summary: Depthwise seperable convolution as a building block of a ConvNet, allows you to carry out inference much more efficiently than using a normal convolution.

Depthwise Separable Convolution

Depthwise Convolution

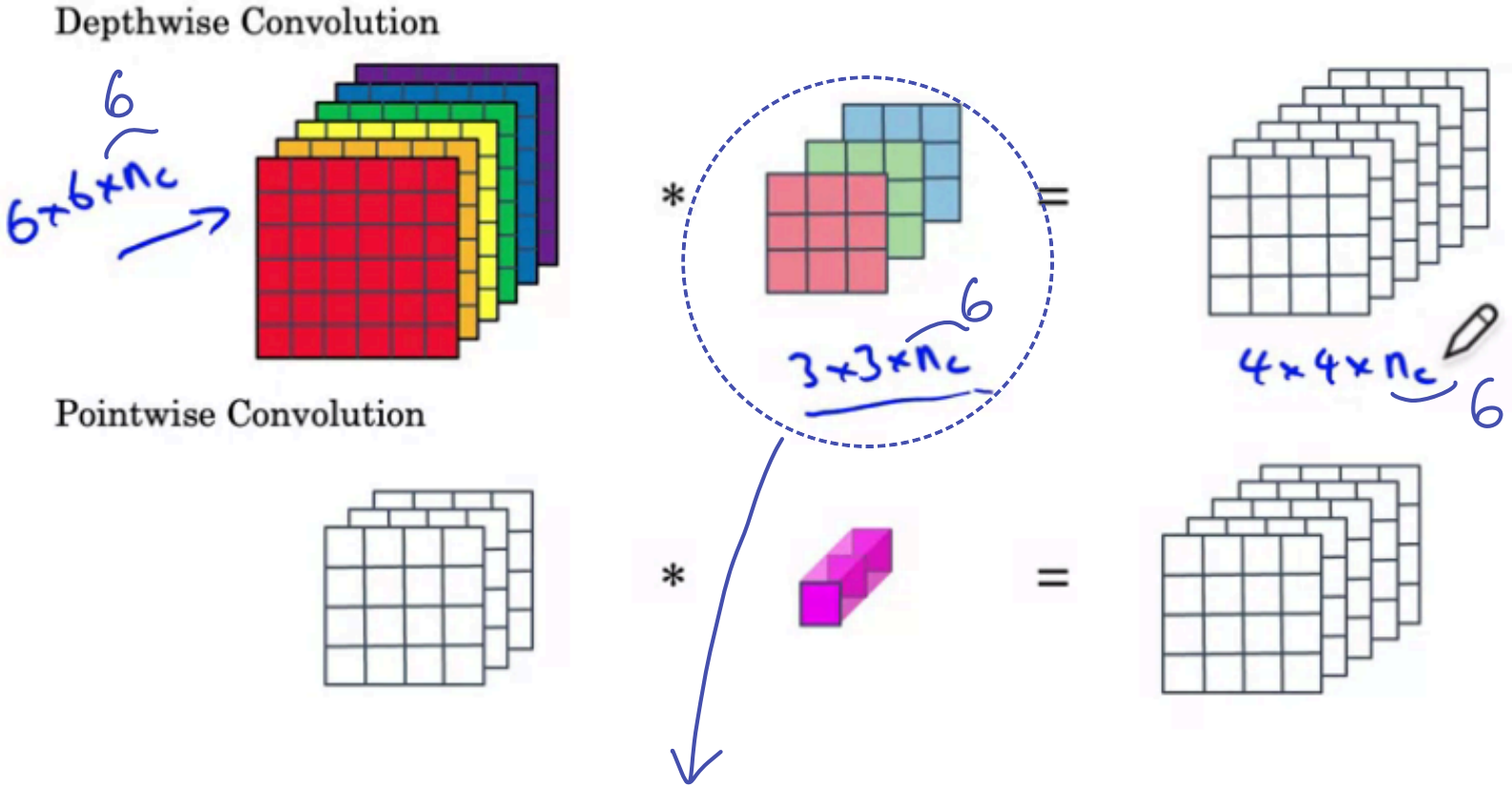


Pointwise Convolution



Andrew Ng

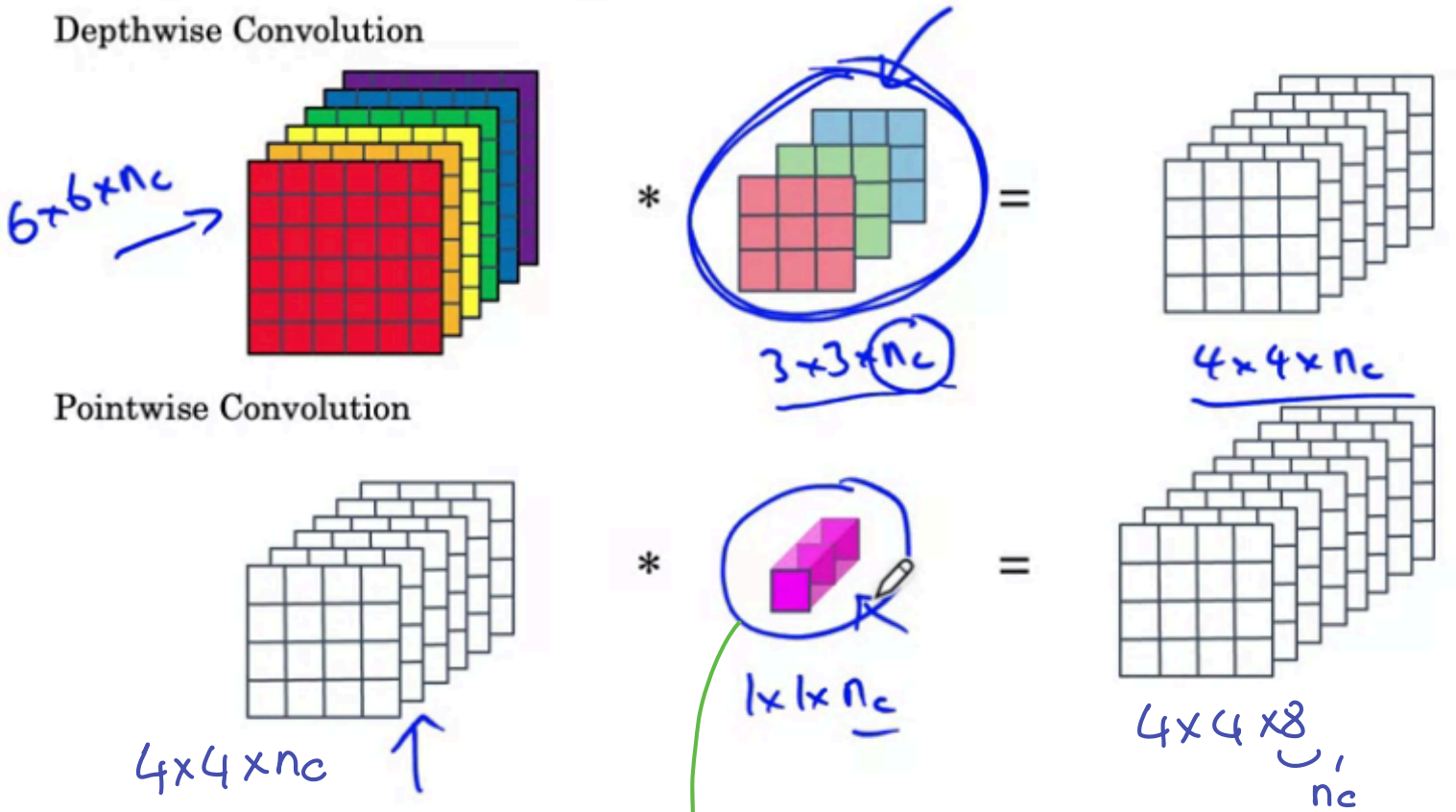
Depthwise Separable Convolution



This diagram of stack of 3 filters will represent 3x3x6 filters for notational simplicity. (icon to represent depth-wise separable convolution)

Andrew Ng

Depthwise Separable Convolution



Andrew Ng

this icon will be used represent 1x1xnc filter for simplicity. (Point-wise convolution)

deeplearning.ai

Convolutional Neural Networks

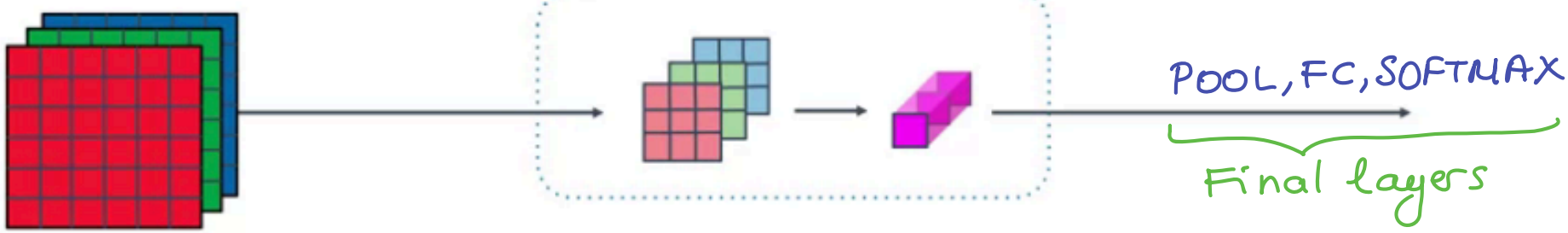
MobileNet Architecture

- Goal: Put the depth-wise seperable convolution into a neural network in order to build MobileNet.

MobileNet

MobileNet v1

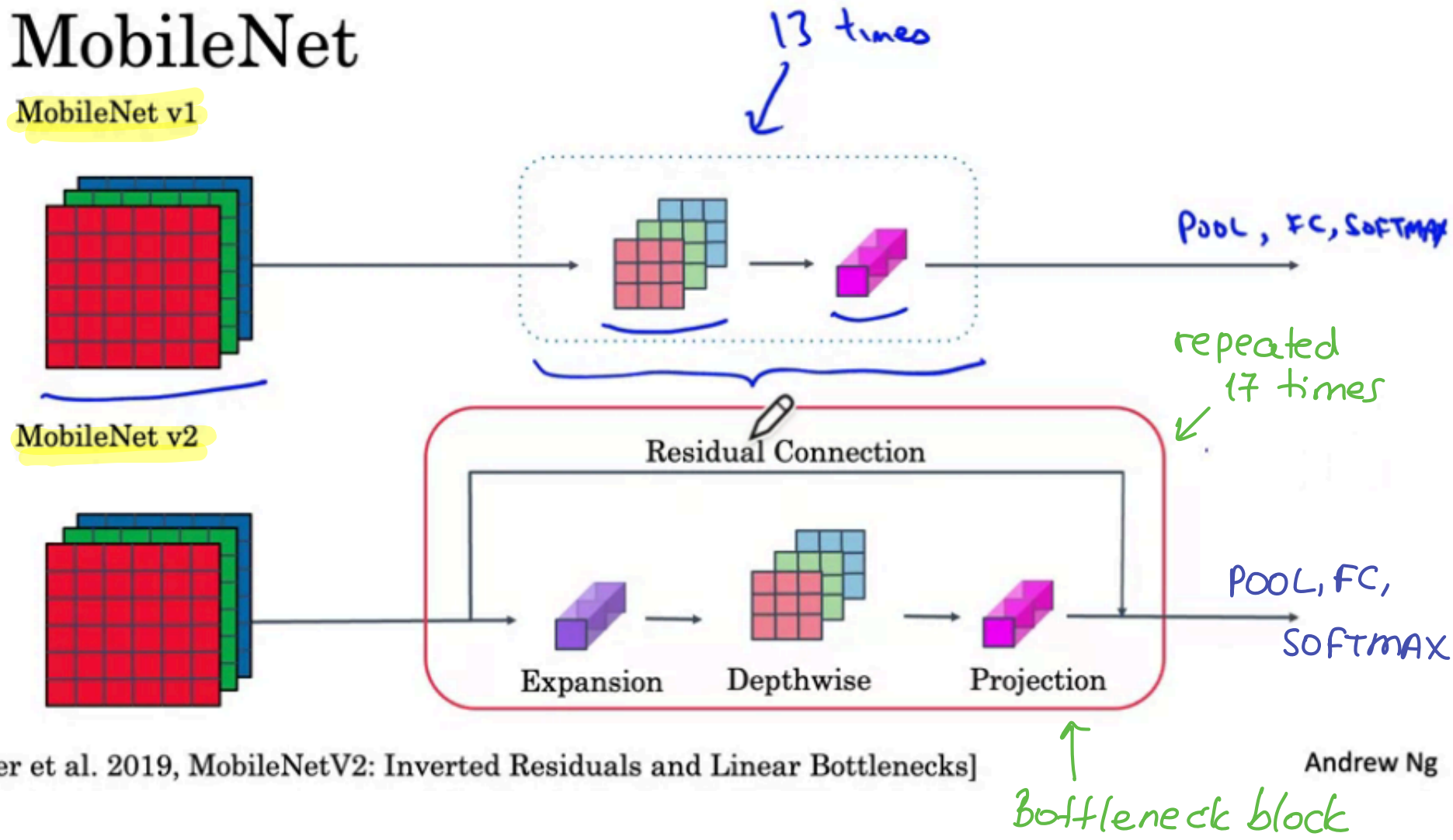
MobileNet v1 used this block 13 times for image classification.



Idea: Everywhere that you previously had an expensive convolution operation, use a much less expensive depthwise separable convolution operation.

Andrew Ng

MobileNet



[Sandler et al. 2019, MobileNetV2: Inverted Residuals and Linear Bottlenecks]

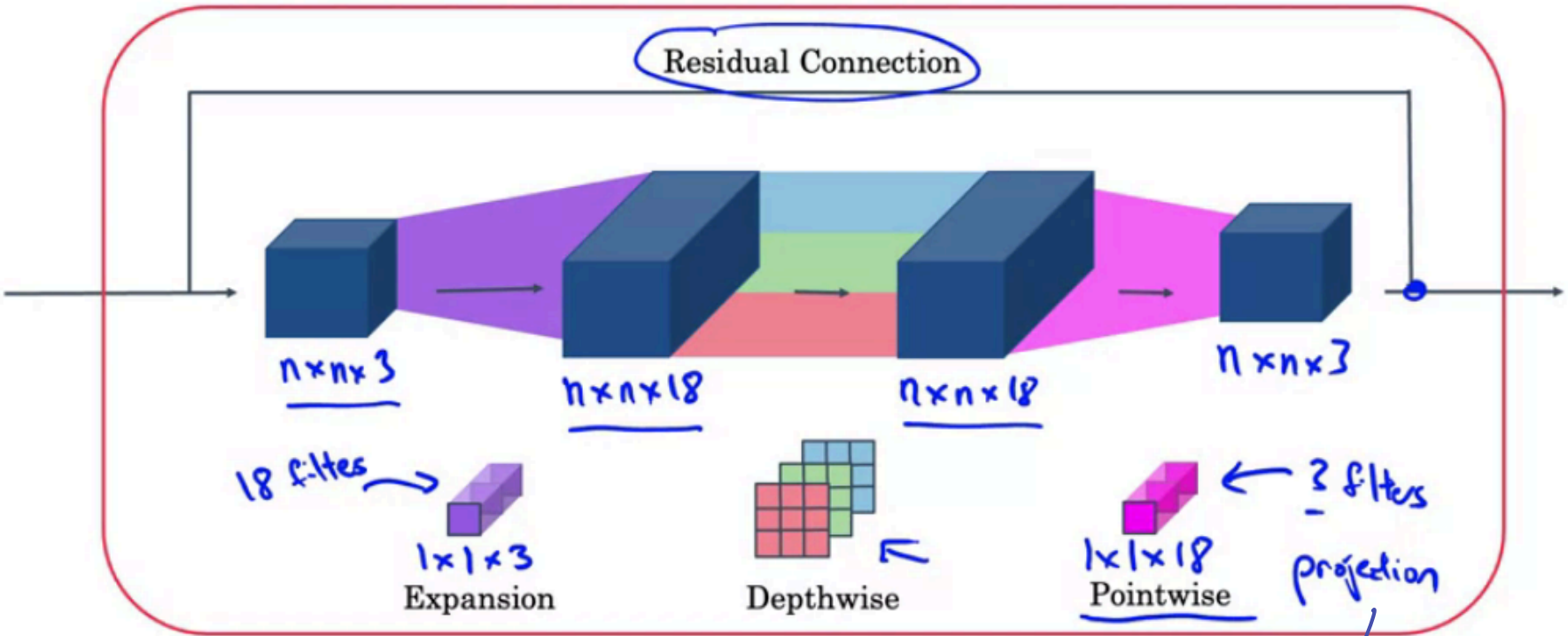
Andrew Ng

In MobileNet v2 there were 2 extensions to v1:

1) A residual connection (skip connection) was added which takes the input from the previous layer and passes it directly to the next layer (this allows ingredients to propagate backward more efficiently.
2) Expansion layer

- MobileNet v2 repeats the red block 17 times, and finally uses pooling, fully-connected and softmax layers for classification.
- main idea of MobileNet: repeated blocks reduce computational cost.

MobileNet v2 Bottleneck



[Sandler et al. 2019, MobileNetV2: Inverted Residuals and Linear Bottlenecks]

Andrew Ng

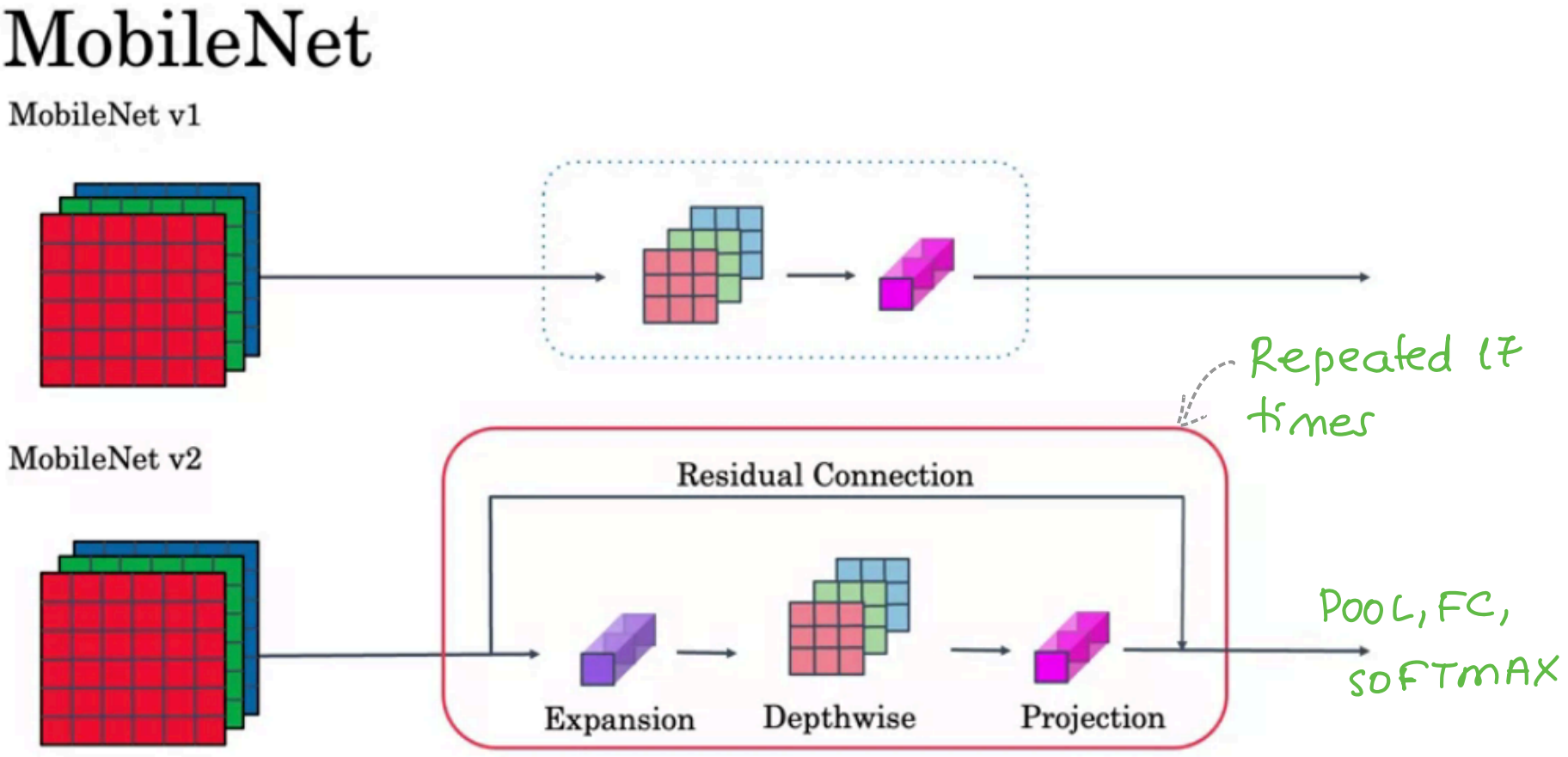
projecting from n×n×18 to n×n×3

- Why do we need the bottleneck block? The bottleneck block accomplishes two things:
  (1) By using the expansion operation, it increases the size of the representation within the bottleneck block. This allows the neural network to learn a richer function.

  (2) When deploying on a mobile device, or edge device, there will be heavy memory constraints. The bottleneck block user the pointwise convolution (i.e. projection operator) in order to project it back down to a smaller set of values, so that when you pass this to the next block, the amount of memory needed to store these values is reduced back down.

- Summary: Bottleneck block enables a richer set of computations, thus allow your NN to learn richer and more complex functions, while also keeping the amount of memory, i.e. the size of activations that you pass from layer to layer, relatively small.
  - That's why MobileNet v2 can get a better performance than MobileNet v1, while still using only a modest amount of compute and memory resources.

MobileNet



[Sandler et al. 2019, MobileNetV2: Inverted Residuals and Linear Bottlenecks]

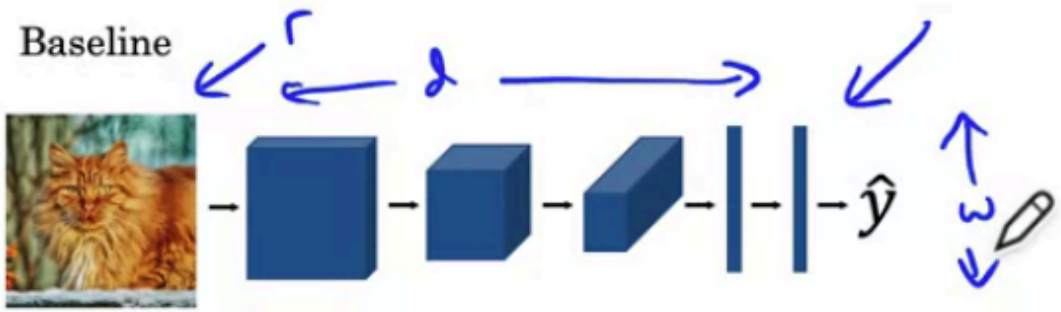
deeplearning.ai

Convolutional Neural Networks

EfficientNet

- MobileNet V1 and V2 described a way to implement a neural network, that is more computationally efficient.
- Is there a way to tune MobileNet, or some other architecture to your specific device?
- Maybe implementing a computer vision algorithm for different brands of mobile phones with different amounts of compute resources, or different edge devices.
- If you have more compute power, you may use a slightly bigger network to get a bit more accuracy.
- Or if you're computationally constrained, you may want a smaller network that runs a bit faster, at the cost of a little accuracy drop.
- How can we automatically scale up or down neural networks for a particular device?
- EfficientNet gives a way to do so.

EfficientNet



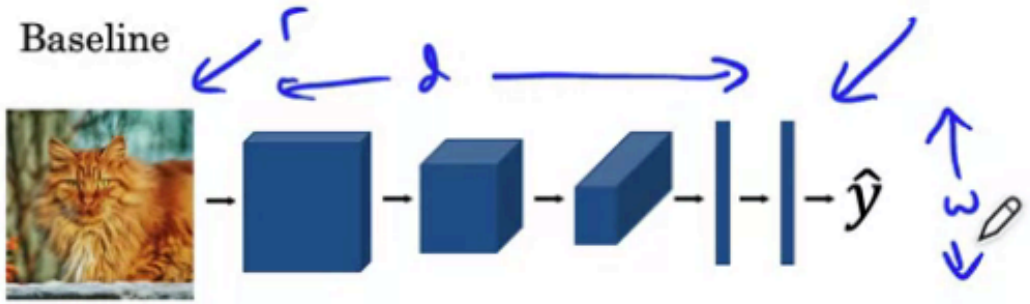
- Baseline neural network structure, where
  - r: input image resolution
  - d: depth of the network
  - w: width of the layers

  } 3 things you could change to scale up or down.

[Tan and Le, 2019, EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks]

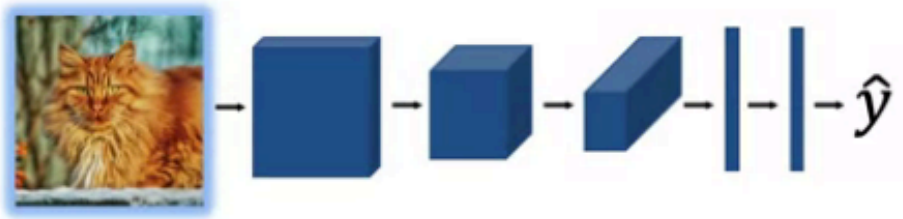
EfficientNet

Baseline



r: resolution of the input image could be increased.

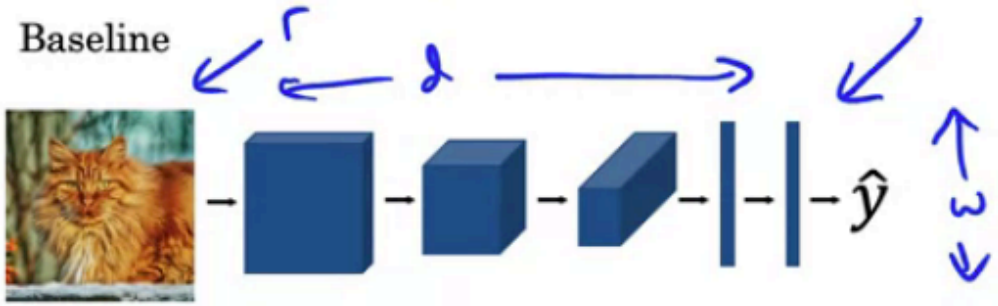
Higher Resolution



[Tan and Le, 2019, EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks]

EfficientNet

Baseline

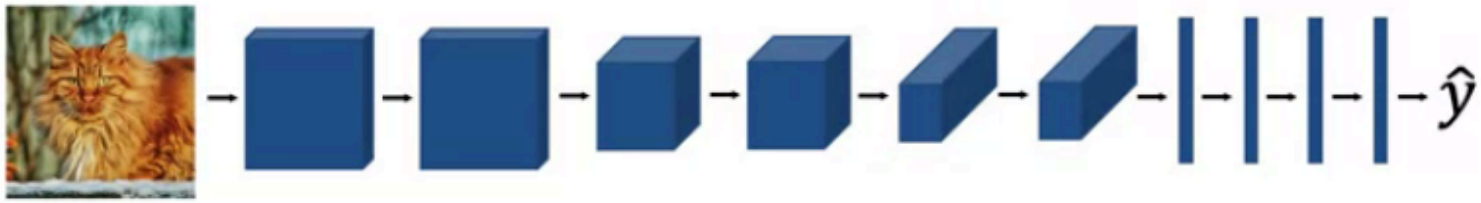


r resolution

d depth

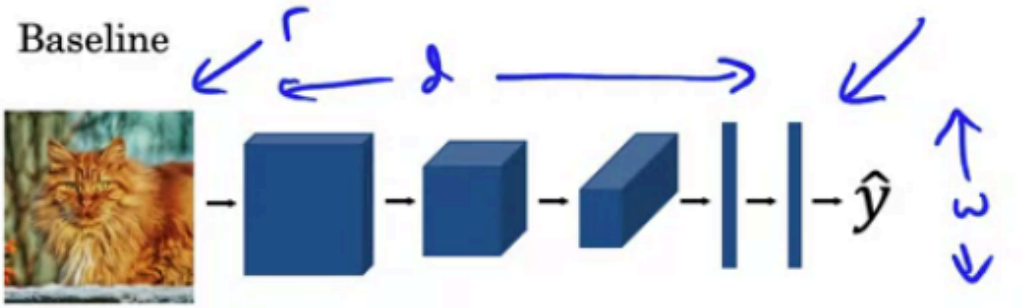
'increase the depth of the network

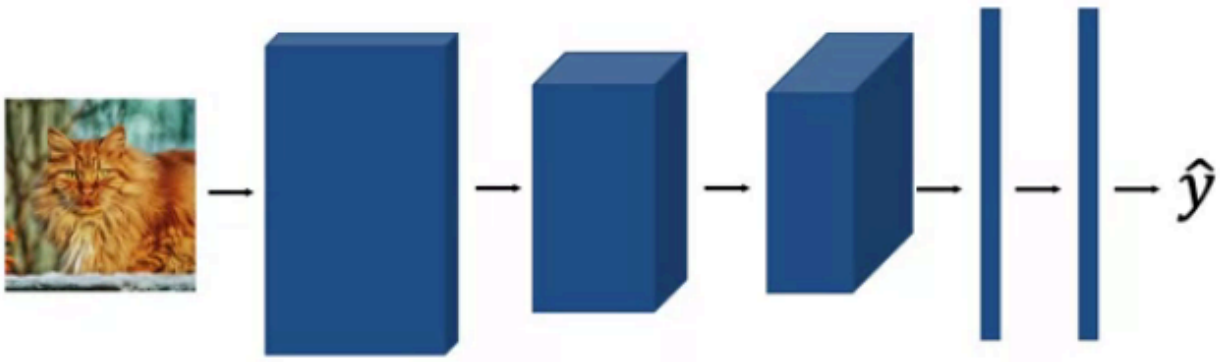
Deeper



[Tan and Le, 2019, EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks]

EfficientNet





r resolution
d depth
w width

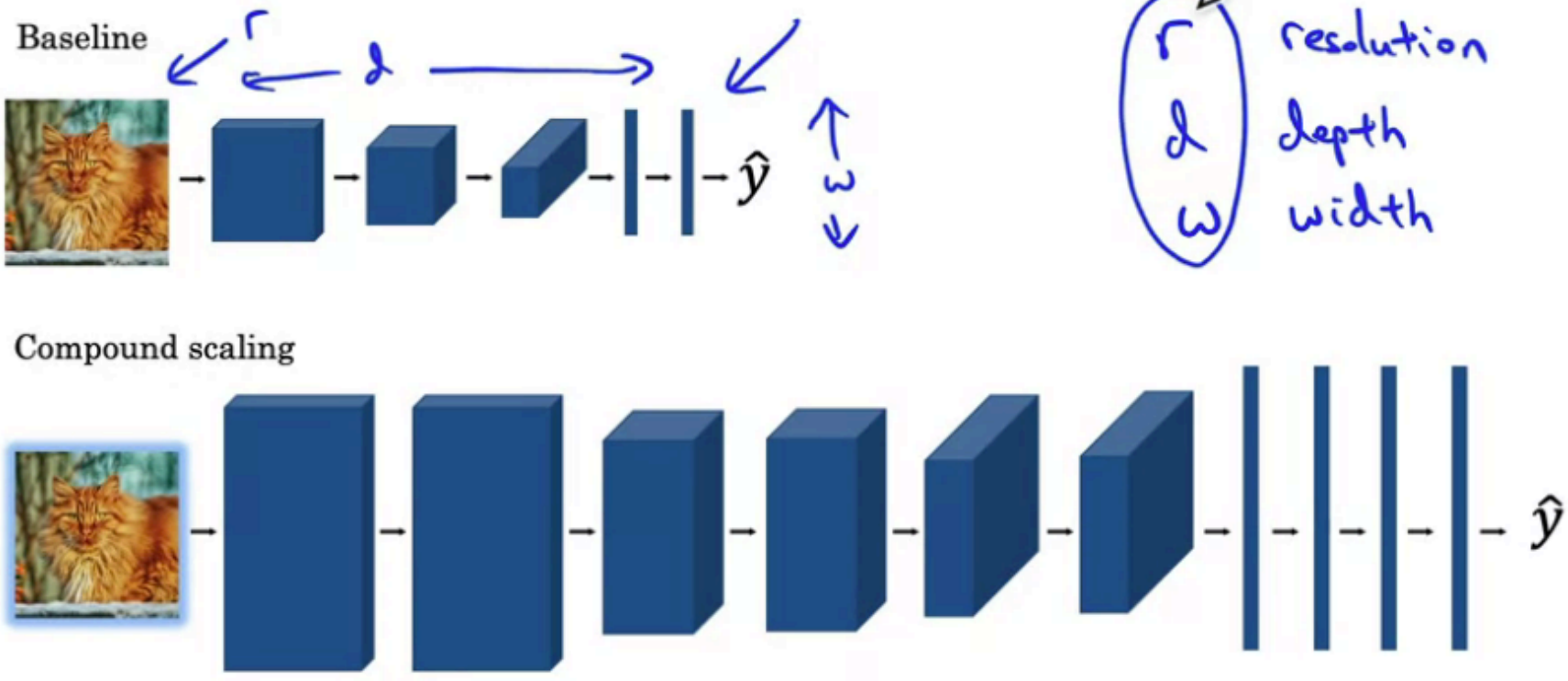
increase the width of the layers.

[Tan and Le, 2019, EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks]

Compound Scaling:

All of them could be increased given the computational resources.

EfficientNet



[Tan and Le, 2019, EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks]

Question: If you want to scale up r, d and w, what's the rate at which you should scale up each of these?

- Should you double the resolution and leave depth the same?
- Should you double the depth and leave others the same?
- Increase resolution by 10%, increase depth by 50%, and width by 20%?

- What's the best trade-off between r, d and w, to scale up or down your neural network to get the best possible performance within your computational budget?

- If you're looking to adapt a neural network architecture for a particular device, look at one of the open-source implementations of EfficientNet, which will help you to choose a good trade-off between r, d and w.

deeplearning.ai

Practical advice for using ConvNets

Using open-source implementations

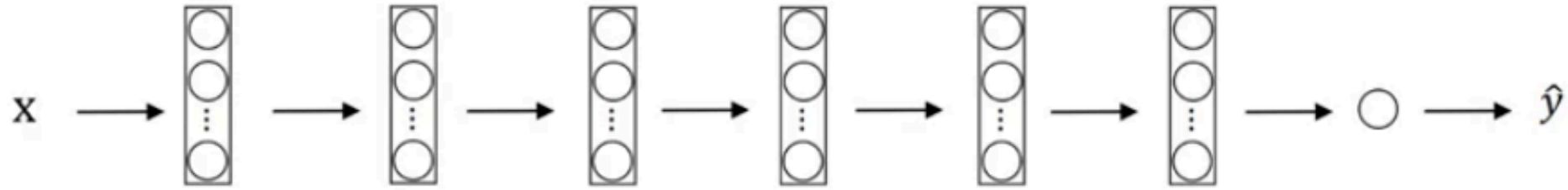
- We've learned about several highly effective neural network and ConvNet architectures.
- These networks are difficult to replicate because of a lot of details about tuning of the hyperparameters such as learning decay etc.
- If you see a research paper that whose results you would like to build on top of, first look for an open-source implementation (by the authors).
  - Download from Github.
  - These networks take a long time to train (weeks or months), and someone else might have used multiple GPUs and a very large dataset to pretrain some of these networks. Has done a painful hyper-parameter seach process, as well.
  - You can do transfer learning rather than implementing them from scratch.
  - In computer vision rather than training CNNs from scratch, from random initialization, you often make much faster progress if you download CNNs that someone else has already trained on the network architecture and use that as pre-training and transfer that to a new task you are interested in.
  - Then, contribute back to the open-source community.

deeplearning.ai

Practical advice for using ConvNets

Transfer Learning

Transfer Learning

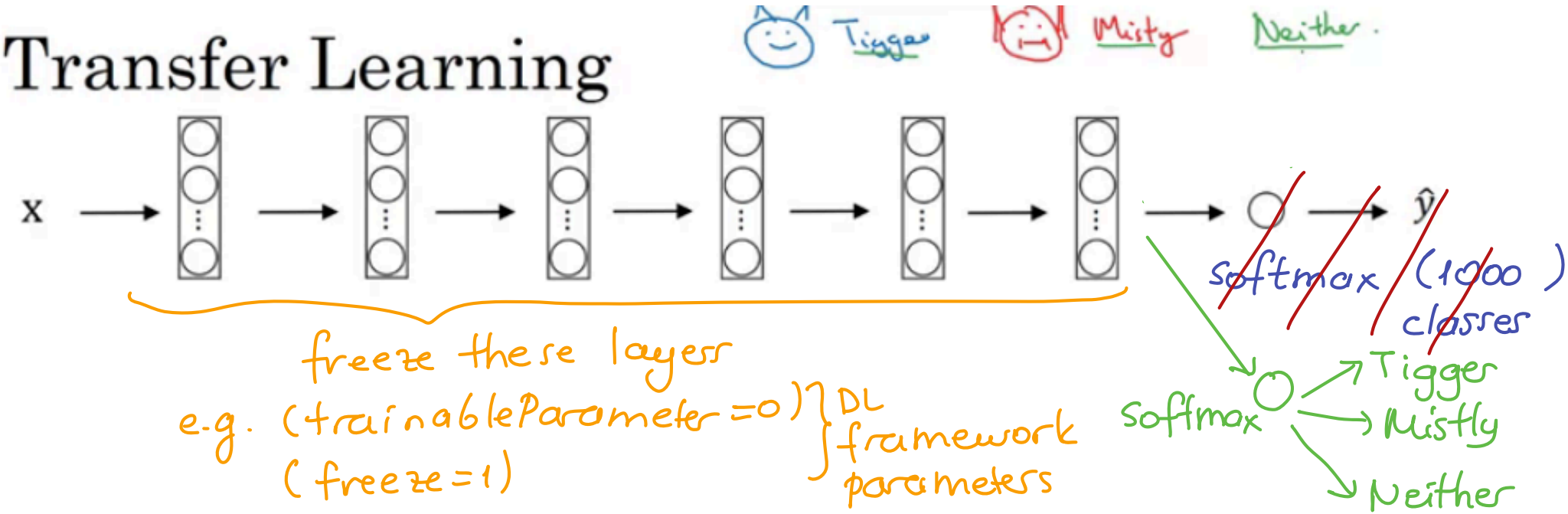


Problem: Build a cat detector to recognize your own pet cat. You have two cats: Tigger and Misty. 3 classes: Tigger, Misty, Neither.

- You don't have a lot of pictures of Tigger or Misty, so your training set will be small.
- What can you do?
- You can go online and download some open-source implementation of a neural network (code and weights).
- There are a lot of networks you can download that have been trained on e.g. ImageNet data set, which has 1000 different classes.

Andrew Ng

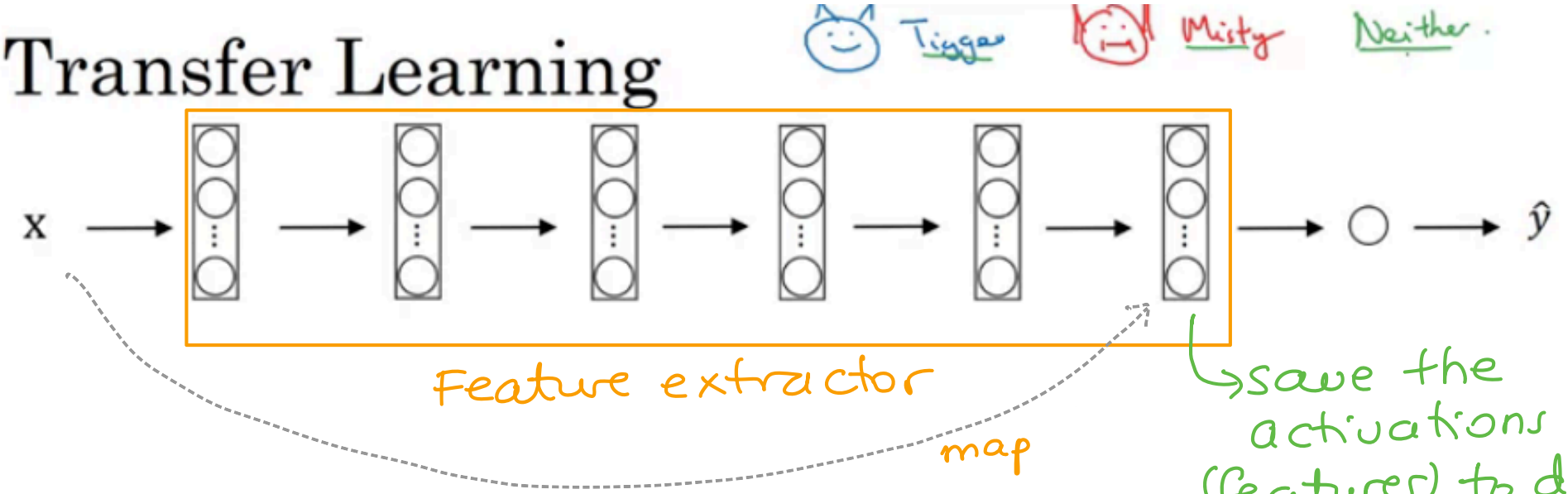
Transfer Learning



- Delete the softmax layer for 1000 classes and create your own softmax layer that outputs 3 classes: Tigger, Misty, Neither.
- Parameters of previous layers might be frozen. Just train the parameters associated with the softmax layer.
- By using someone else's pretrained weights you may get pretty good performance even on a small dataset.

Andrew Ng

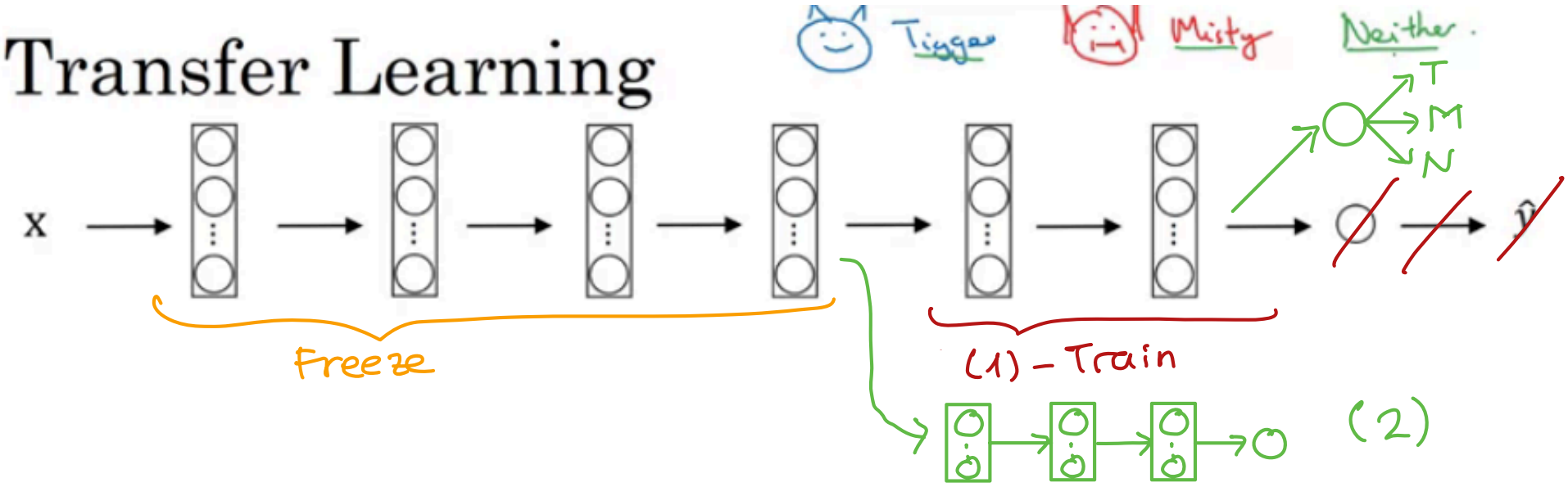
Transfer Learning



- Another way: Since all the early layers are frozen, these layers take the input image and map it to some set of activations in the final layer.
  - Pre-compute the features or activations from that layer and save them to disk (from all training samples).
  - Then, train a shallow softmax model from this feature vector to make a prediction.
  - Saving to disk is done in order not to re-compute the feature vectors every time you make an epoch (a pass) thru the training set.

Andrew Ng

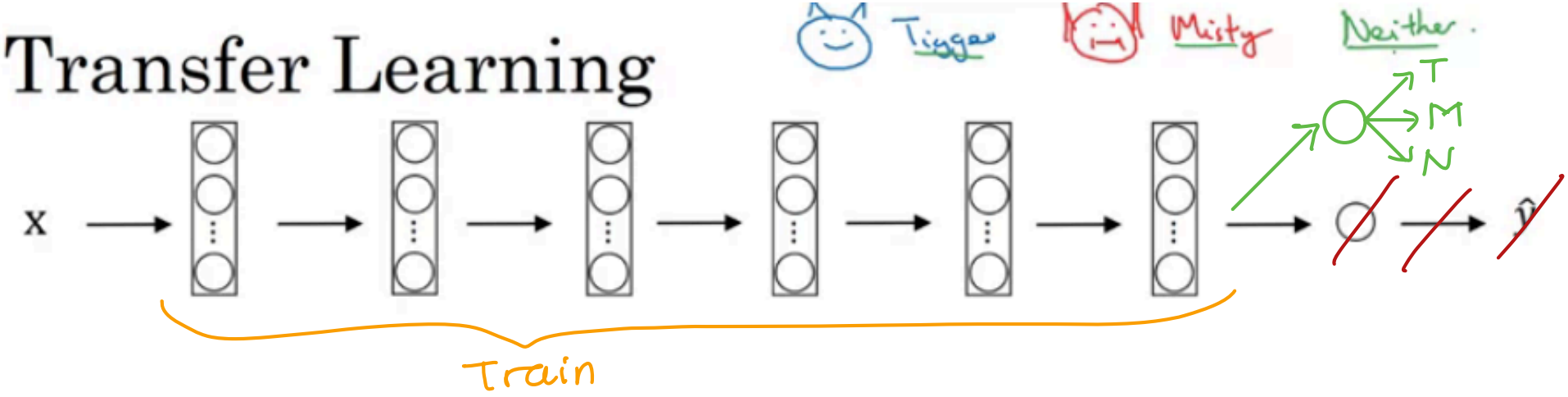
Transfer Learning



- What if you have a larger training set? (many pictures of Tigger, Misty, Neither).
- Freeze fewer layer. Train the other layers. Change the softmax layer.
  - (1) - The trained layers may continue training starting with pretrained weights.
  - (2) - OR You may insert new hidden layers, which are randomly initialized.

Andrew Ng

Transfer Learning



- If you have a lot of data, use the whole network weights as initialization and train the whole network.

Andrew Ng

deeplearning.ai

Practical advice for using ConvNets

Data augmentation

- Most computer vision tasks could use more data.
- Data augmentation is one of the techniques that is often used to improve the performance of computer vision systems.
- Computer vision is a complicated task. You need to learn a fairly complicated function. More data will help in most applications.

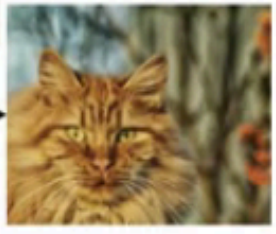
Common augmentation method

- Mirroring should preserve whatever you're trying to recognize.

Mirroring

y

Random Cropping



Randomly cropped region should not be too small.

Rotation

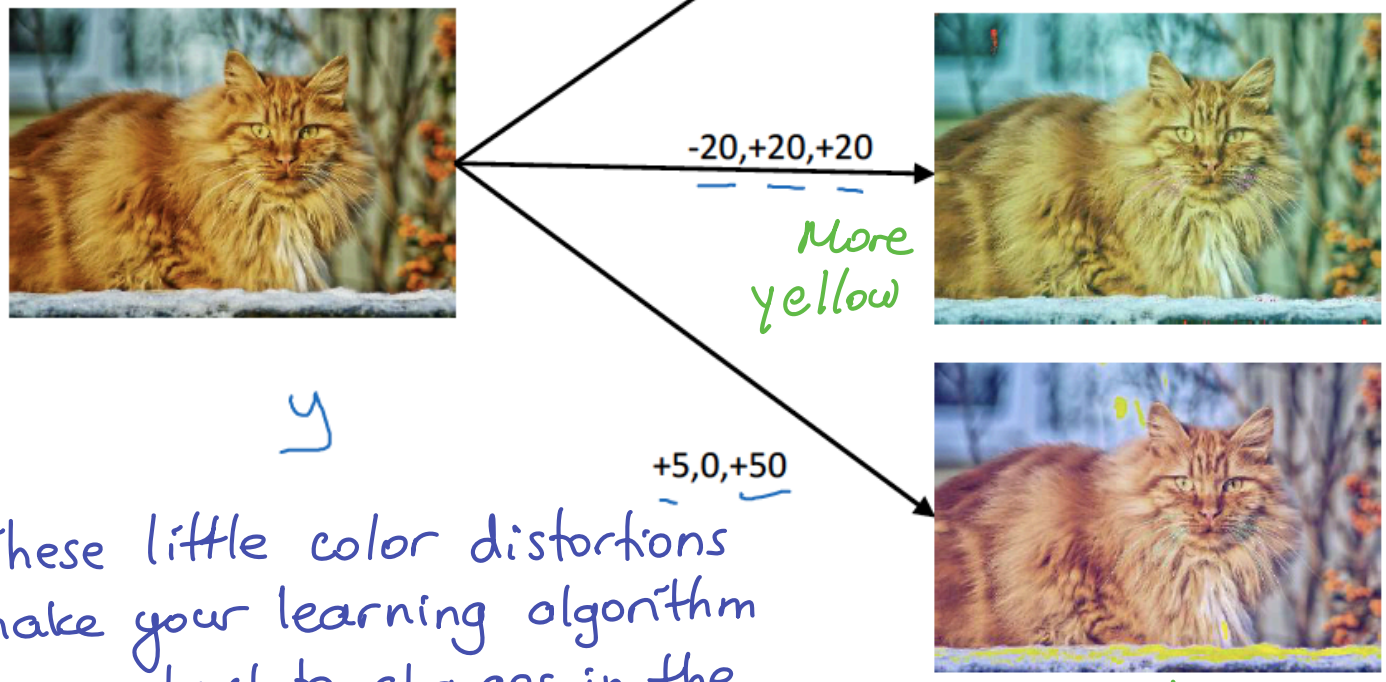
Shearing

Local warping

...

Andrew Ng

Color shifting



More purple

More yellow

more blue

content of the image is the same

- These little color distortions make your learning algorithm more robust to changes in the colors of your images.

Advanced: Method

PCA

ml-class.org

[AlexNet paper

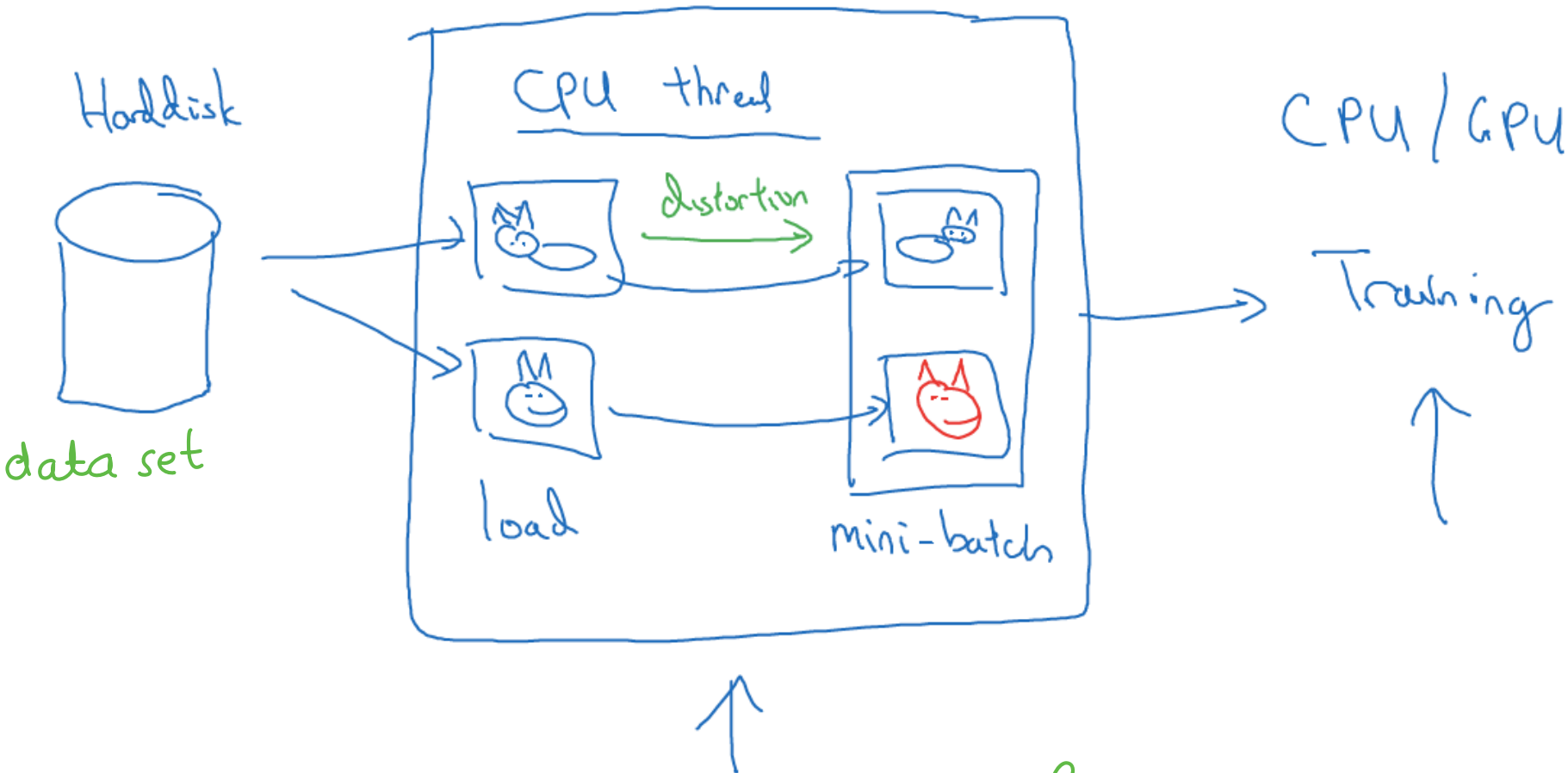
["PCA color augmentation."]

R B G

- If the image is mainly purple, if it has mainly red and blue tints, and very little green, it tries to keep the overall color of the tint the same.

Andrew Ng

Implementing distortions during training



One thread is resposible for loading data and implementing distortions. and passes them to another thread for training. They can run in parallel.

Andrew Ng

- Data augmentation process also has a few hyperparameters such as how much color shifting to implement and which parameters to use for random cropping?

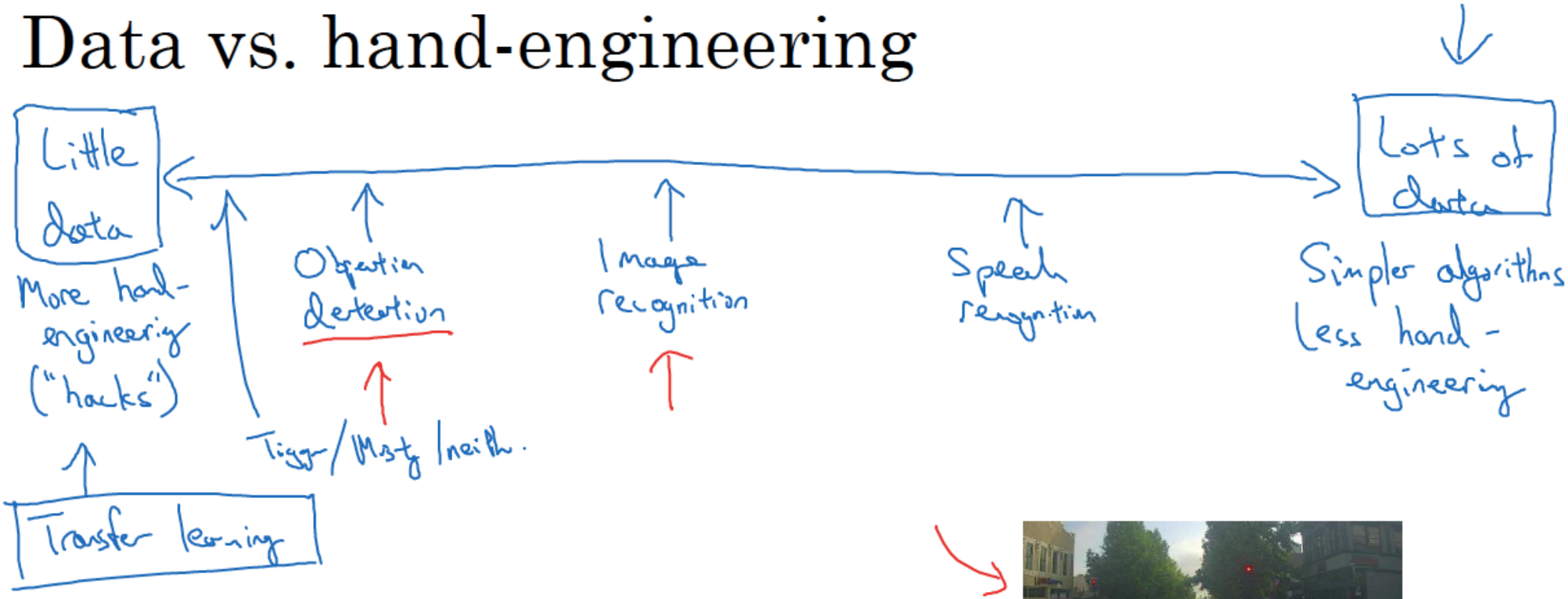
- A good place to start might be to use someone else's open-source implementation.

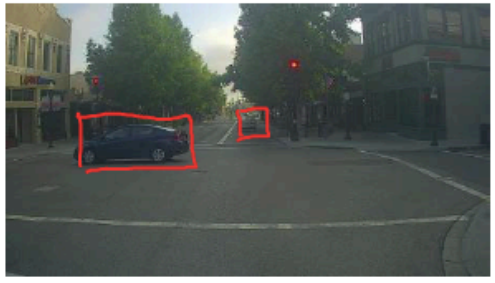
deeplearning.ai

Practical advice for using ConvNets

The state of computer vision

Data vs. hand-engineering





Two sources of knowledge

- Labeled data (x, y)
- Hand engineered features/network architecture/other components

Count on this if you have little data

Andrew Ng

- Deep learning has been applied to computer vision, NLP, speech recognition, online advertising, logistics etc.

- most machine learning problems fall somewhere on the spectrum between where you have relatively little data to where you have lots of data.

- Object detection: There exists relatively less data for this problem since it is more costly to label the images with bounding boxes around each object.

- If you have a lot of data, there's less need for hand-engineering the features. Just use a big network and let the algorithm learn.
- If you have little data, hand-engineering of features is the best way to get good performance. Hand-engineering requires insight into the problem.
    - Hand-engineering of network architecture is also very important in computer vision. (little data)
    - Transfer learning helps a lot if you have relatively little data.

Tips for doing well on benchmarks/winning competitions

Ensembling

3-15 networks

$\rightarrow \hat{y}$

- Train several networks independently and average their outputs

Multi-crop at test time

- Run classifier on multiple versions of test images and average results

10-crop

mirror image

1 crop (blue region is cropped and tested) + 4 corners crop + 1 Crop + 4 corners crop = 10 crops

Andrew Ng

- There's a lot of attention on doing well on benchmarks/competitions.
- It helps the community to figure out what are the most effective algorithms.
- You also see in the papers people do things that allow you to do well on a benchmark, but you wouldn't really use in a production or a system that you deploy in an actual application.
    - Ensembling: train several networks and average their outputs. This increases accuracy but slows down the performance. Hence, not generally used in production if the computational budget is not large.
    - Multi-crop at test time.

Use open source code

- Use architectures of networks published in the literature
- Use open source implementations if possible
- Use pretrained models and fine-tune on your dataset

Andrew Ng